A Hybrid Multi-Column Convolutional Neural Network and YOLOv8 Framework for Real-Time Crowd Density Estimation and Behavioural Analysis

21CSP302L – PROJECT

Submitted by

VIKRAM S [RA2311032010XXX]
POORVIKHA S [RA2311032010YYY]

Under the Guidance of

Dr. Nallarasan V
Assistant Professor, Department of Networking and Communications

in partial fulfillment of the requirements for the degree of

BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE ENGINEERING
with specialization in Internet of Things

DEPARTMENT OF NETWORKING AND COMMUNICATIONS
COLLEGE OF ENGINEERING AND TECHNOLOGY
SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
KATTANKULATHUR – 603 203

MAY 2026

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
KATTANKULATHUR – 603 203

BONAFIDE CERTIFICATE

Certified that 21CSP302L – Project report titled "A Hybrid Multi-Column Convolutional Neural Network and YOLOv8 Framework for Real-Time Crowd Density Estimation and Behavioural Analysis" is the Bonafide work of "VIKRAM S [RA2311032010XXX] and POORVIKHA S [RA2311032010YYY]" who carried out the project work under my supervision. Certified further, that to the best of my knowledge the work reported herein does not form any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

Dr. Nallarasan V

SUPERVISOR

Assistant Professor

DEPARTMENT OF

NETWORKING AND COMMUNICATIONS

Dr. M. Lakshmi

PROFESSOR & HEAD

DEPARTMENT OF

NETWORKING AND COMMUNICATIONS

Submitted for the project viva-voce held on _____

EXAMINER 1 **EXAMINER 2**

Department of Networking and Communications
SRM Institute of Science & Technology

Own Work Declaration Form

This sheet must be filled in (each box ticked to show that the condition has been met). It must be signed and dated along with your student registration number and included with all assignments you submit – work will not be marked unless this is done.

To be completed by the student for all assessments

Degree / Course : B. Tech Computer Science and Engineering – Internet of Things

Student Name : Vikram S, Poorvikha S

Registration Number : RA2311032010XXX, RA2311032010YYY

Title of Work : A Hybrid Multi-Column Convolutional Neural Network and YOLOv8 Framework for Real-Time Crowd Density Estimation and Behavioural Analysis

We hereby certify that this assessment compiles with the University's Rules and Regulations relating to Academic misconduct and plagiarism, as listed in the University Website, Regulations, and the Education Committee guidelines.

We confirm that all the work contained in this assessment is our own except where indicated, and that we have met the following conditions:

- Clearly referenced and listed all sources as appropriate.
- Referenced and put in inverted commas all quoted text (from books, web, etc.).
- Given the sources of all pictures, data, etc. that are not our own.
- Not made any use of the reports, essays of any other students either past or present.
- Acknowledged in appropriate places any help that we have received from others (e.g. fellow students, technicians, statisticians, external sources).
- Compiled with any other plagiarism criteria specified in the Course handbook / University website.

We understand that any false claim for this work will be penalized in accordance with the University policies and regulations.

DECLARATION

We are aware of and understand the University's policy on Academic misconduct and plagiarism and we certify that this assessment is our own work, except where indicated by referring, and that we have followed the good academic practices noted above.

Vikram S **Poorvikha S**

If you are working in a group, please write your registration numbers and sign with the date for every student in your group.

ACKNOWLEDGEMENTS

We express our humble gratitude to Dr. C. Muthamizhchelvan, Vice-Chancellor, SRM Institute of Science and Technology, for the facilities extended for the project work and his continued support.

We extend our sincere thanks to Dr. Leenus Jesu Martin M, Dean-CET, SRM Institute of Science and Technology, for his invaluable support.

We wish to thank Dr. Revathi Venkataraman, Professor and Chairperson-AI, School of Computing, SRM Institute of Science and Technology, for her support throughout the project work.

We encompass our sincere thanks to Dr. M. Pushpalatha, Professor and Associate Chairperson – CS, School of Computing and Dr. C. Lakshmi, Professor and Associate Chairperson – AI, School of Computing, SRM Institute of Science and Technology, for their invaluable support.

We are incredibly grateful to our Head of the Department, Dr. M. Lakshmi, Department of Networking and Communications, SRM Institute of Science and Technology, for her suggestions and encouragement at all the stages of the project work.

We want to convey our thanks to our Project Coordinators, Panel Head, and Panel Members, Department of Networking and Communications, SRM Institute of Science and Technology, for their inputs during the project reviews and support.

We register our immeasurable thanks to our Faculty Advisor, Dr. Nivedhitha M, Department of Networking and Communications, SRM Institute of Science and Technology, for leading and helping us to complete our course.

Our inexpressible respect and thanks to our guide, Dr. Nallarasan V, Assistant Professor, Department of Networking and Communications, SRM Institute of Science and Technology, for providing us with an opportunity to pursue our project under his mentorship. He gave us the freedom and the space to explore the research topics of our interest, and his weekly feedback on the MCNN, YOLOv8 and ByteTrack integration was the single biggest factor in shaping the final form of this work.

We sincerely thank all the staff members of the Department of Networking and Communications, School of Computing, SRM Institute of Science and Technology, for their help during our project. Finally, we would like to thank our parents, family members and

friends for their unconditional love, constant support and encouragement throughout the project term.

Vikram S **Poorvikha S**

ABSTRACT

Crowd-related incidents during big public events have become a common headline in the last few years, and the older method of putting a few human operators in front of twenty CCTV monitors no longer keeps up with how quickly such situations can escalate. In this project, called CrowdInsight AI, we put together a hybrid pipeline that tries to fix this gap by combining three different ideas from deep learning – an MCNN model for estimating crowd density, YOLOv8 for fast person detection, and ByteTrack for keeping track of people across frames even when they are partly hidden. All three run inside one Python process so the whole thing can sit on an edge device.

On top of this pipeline, we propose a Stampede Risk Index, or SRI for short, which mixes three signals – density, average speed of moving people, and the spread of their walking directions – into one single number between 0 and 1. The weight vector (0.40, 0.35, 0.25) was picked by a small grid search on a held-out split. Whenever the SRI crosses 0.65 for more than five frames in a row, an alert is pushed through four channels at the same time: a console line, a small audio chirp, a webhook POST, and an appended JSONL log entry. The whole system is wrapped in a React-19 single-page dashboard, also called CrowdInsight AI, that talks to the engine through a thin FastAPI server and offers two modes – an Image-Gallery mode for still inputs and a Live-Stream mode for RTSP/USB feeds.

The MCNN backbone was trained from scratch on the ShanghaiTech Part B dataset and we compared the results against MCNN, CSRNet and a YOLOv8-only baseline. On the 316-image test partition we obtained an MAE of 49.72 and an RMSE of 80.32. The end-to-end pipeline runs at about 17.4 FPS on a 720 p stream on an NVIDIA RTX 3060, and the weighted F1 across the four supported behavioural events – surge, panic, loitering, intrusion – is 0.88. The whole pipeline has about 3.5 million parameters, which is roughly one-fifth of CSRNet alone, making it small enough to fit on devices like the Jetson Orin Nano. Our ablation study shows that the SRI fusion alone adds about 0.09 F1 over the untracked density-plus-detection version, which is what made us treat SRI as the main contribution of the work.

The rest of this report walks through the related work, the dataset and the way we built the ground-truth density maps, the two-sprint engineering plan we followed, the system requirements, the testing strategy and finally the actual results we obtained. It also lists six follow-up directions that we believe would push this baseline further, from swapping the

backbone for something lighter to predicting the SRI ahead of time using a small temporal model.

Keywords: Crowd density estimation, MCNN, YOLOv8, ByteTrack, stampede prediction, behavioural analytics, edge deployment, public safety, real-time surveillance, Stampede Risk Index.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

CHAPTER 1
INTRODUCTION

This chapter sets the stage for the rest of the report. We start by explaining why crowd analysis has slowly turned into a must-have feature for any modern public space, then walk briefly through how the related computer-vision methods evolved into what we use today. After that we state the gap our work tries to fill, the reasons we picked a multi-network design, and finally map the project to a few of the United Nations Sustainable Development Goals.

1.1 Introduction to the Project

Public events – religious processions, IPL matches, concerts, political rallies and even crowded metro stations – are happening more often and at much larger scales than they did even ten years ago. Sadly, crowd-crush incidents and stampede deaths have grown with them. Studies from disaster-management bodies and the WHO show that more than seventy percent of crowd-related fatalities happen inside the first ninety seconds of a disturbance, which is well before any human guard can dispatch help. The traditional way of dealing with this is to put one or two operators in front of a wall of CCTV monitors, sometimes twenty or more feeds at once. After an hour or so of staring at the screens, fatigue sets in, attention drifts, and the operator misses exactly the kind of rapid changes that lead to a stampede. So the human-in-the-loop approach is not really enough by itself anymore.

On the brighter side, computer vision and deep learning have matured a lot in the same period. Person detectors such as YOLOv8 [4] now hit decent mAP numbers on the COCO benchmark and still run at more than a hundred frames per second on a mid-range GPU. Trackers like ByteTrack [3] can keep an ID stable even when the person is partially blocked. Density estimators such as MCNN [1] and CSRNet [2] count thousands of people from a single image by predicting a continuous density map and integrating it, instead of trying to draw a box around every single head.

Even with all this progress, real surveillance setups in the field still feel disconnected. One server runs detection, another might run tracking, and density estimation is rarely used at all. The CrowdInsight AI project tries to bring all three of them together – density (MCNN), detection (YOLOv8) and tracking (ByteTrack) – inside one low-latency Python pipeline. The combined output is then squeezed into a single number between 0 and 1 that we call the Stampede Risk Index, or SRI. The SRI takes density, average movement speed and how spread

out the directions are, and uses that to drive an automatic alert. On the deployment side, the whole thing is shipped as a Python engine with a small React dashboard, and we kept the model footprint small enough that it can be moved onto edge devices like the NVIDIA Jetson Orin Nano.

1.2 Problem Statement and Description

After surveying both the academic literature and a few real deployments, we found five recurring problems that stop existing systems from being useful for actual stampede prevention. First, detector-only methods systematically miss people in dense regions because heads keep blocking each other. Second, density-estimation models give us only a heatmap and a number, but they don't know who is who, so they can't pick out the specific group that is moving fast. Third, the few research papers that do try to combine detection with density end up using heavy backbones such as ResNet-101 or Swin-Transformer, which simply will not run on the kind of hardware a campus or a small venue can actually afford. Fourth, almost every paper we looked at reports only the counting accuracy – nobody tells us how often a panic alert misfires, or how many seconds early it is. Fifth, even when a working prototype exists, it usually lives inside a Jupyter notebook. There is rarely a clean UI that a security operator who is not a Python programmer can actually use.

Our project tries to attack all five of these issues at once. We pair MCNN with YOLOv8 so that the dense regions get covered by the density branch while the sparse regions are handled by the detector. ByteTrack is hooked onto the YOLOv8 output so every visible person gets a stable ID, which lets us compute speed and trajectory features. We deliberately kept the entire pipeline below sixteen million combined parameters so it stays real-time on a 6 GB GPU. The behavioural events – surge, panic, loitering and intrusion – are tested separately, with their own F1 numbers, so accuracy is not just measured by counting MAE. And finally the engine is wrapped inside a React-19 dashboard called CrowdInsight AI, which any operator can open in a browser.

1.3 Motivation

Three different reasons pushed us to take up this project. The first one is simply humanitarian. Between 2000 and 2024, India alone has lost more than three thousand people to crowd-crush events. If a low-cost system can sit on an edge device and raise an alert about ninety seconds before things go out of hand, that is enough lead time for guards and medical teams to act. Even a small percentage of these incidents prevented per year would be a worthwhile outcome.

The second reason is academic. Density estimation as a problem feels almost solved at this point – the MAE numbers in the literature have plateaued and small improvements are getting harder. But the question of how to combine density, detection, tracking and behaviour into one usable system is still wide open. We felt that a careful engineering job here would produce something where the whole is more useful than the sum of its parts.

The third reason is just engineering curiosity. Edge accelerators like the Jetson Orin Nano, the Hailo-8 and the Coral Edge TPU have made the deployment side much easier than it used to be. The bottleneck has shifted from raw compute to software that actually targets these chips. We wanted to build a system that is friendly to ONNX and TensorRT export from day one, so that moving it to such hardware later is mostly a matter of configuration and not a full rewrite.

1.4 Sustainable Development Goals of the Project

CrowdInsight AI lines up with a handful of the United Nations Sustainable Development Goals. We list the most directly relevant ones below.

- SDG 11 – Sustainable Cities and Communities: a real-time crowd monitoring tool helps city operators keep an eye on bus stations, railway platforms, religious sites and stadia. By warning about overcrowding before it spirals, the system feeds directly into safer urban planning.
- SDG 9 – Industry, Innovation and Infrastructure: we show that high-end deep-learning workloads can run on cheap, embedded hardware. This is especially useful for emerging economies where data-centre style deployments are not affordable.
- SDG 16 – Peace, Justice and Strong Institutions: the JSONL event log keeps a time-stamped, append-only record of every alert. After an incident, this log makes it much easier for investigators to reconstruct what actually happened.
- SDG 3 – Good Health and Well-Being: every stampede that gets stopped early is a real life saved. The webhook channel is designed so that the SRI alerts can be plugged into hospital and ambulance dispatch systems.
- SDG 4 – Quality Education: as a final-year capstone, the work produces an IEEE paper, this report, a Jupyter notebook and a documented codebase. Future students at our department can pick this up as a starting point instead of building from scratch.

To wrap up, this chapter has explained why we worked on this problem, what is missing in existing solutions, and how the project fits into the larger SDG framework. The next chapter looks at the related research in some detail and points out the specific gaps that motivated our design choices.

CHAPTER 2
LITERATURE SURVEY

This chapter walks through the existing work on automated crowd analysis, broken up into three loose groups – density estimation, detection-based counting and tracking-plus-behaviour analysis. Once we are done with the survey, we list the gaps that we feel the literature has left open, which then become the research objectives that drive the rest of the project. The chapter closes with our product backlog (written as user stories) and a phase-by-phase plan of action for the semester.

2.1 Overview of the Research Area

Crowd analysis started off as a small offshoot of pedestrian detection in the early 2000s. The earliest methods were almost entirely hand-engineered: head-shoulder Haar cascades, HOG (Histogram of Oriented Gradients) descriptors, simple motion energy maps. These worked reasonably for sparse scenes but fell apart in crowded ones because heads kept blocking each other, and they were also very sensitive to lighting. Things really shifted with the MCNN paper by Zhang et al. [1] in 2016, which used a multi-column CNN to predict a continuous density map whose integral gives the crowd count – effectively skipping the need to localise each individual. CSRNet [2] then came in 2018 and made the receptive field bigger by using dilated convolutions, and a string of later works (SANet, ScaleNet, BL, DM-Count) tweaked the loss and the supervision targets.

On a parallel track, the detection community kept producing stronger architectures – Faster R-CNN, SSD, RetinaNet, the whole YOLO family and most recently YOLOv8 from Ultralytics [4]. Person detection in real time is now a fairly solved problem in moderate densities. Tracking has come a long way as well. ByteTrack [3] from ECCV 2022 showed that you can squeeze a lot of extra performance out of tracking just by being smarter about low-confidence detections and adding a second matching pass based on IoU. What we found striking, though, is that almost nobody puts these three pieces – density, detection and tracking – into one deployable pipeline. That gap is exactly what CrowdInsight AI tries to close.

2.2 Existing Models and Frameworks

Table 2.1 lists the models we benchmarked or referenced during the design of CrowdInsight AI. It roughly covers the past decade from the original MCNN to YOLOv8.

Table 2.1: Chronological comparison of major crowd-analysis models

Reference	Method	Key Feature	Accuracy/MAE	Limitation
Zhang et al. (2016)	MCNN [1]	Multi-column receptive fields	MAE 26.4 (Part B)	No tracking, no behaviour
Li et al. (2018)	CSRNet [2]	Dilated convolutions	MAE 10.6 (Part B)	Heavy: 16 M params
Idrees et al. (2013)	Multi-source counting [6]	Hand-crafted Fourier	MAE ~36	Pre-deep-learning
Liu et al. (2018)	DecideNet	Detection-density fusion	MAE 21.5	Server-class GPU only
Sam et al. (2017)	SwitchCNN	Patch-level routing	MAE 21.6	Slow router network
Ultralytics (2023)	YOLOv8 [4]	Anchor-free detection	mAP@50 0.93 (person)	Fails in dense crowds
Zhang et al. (2022)	ByteTrack [3]	Two-pass IoU association	MOTA 80.3	Depends on detection quality
Proposed	CrowdInsight AI	MCNN+YOLOv8+ByteTrack+SRI	MAE 49.7 / SRI 0.94 F1	Lightweight trade-off

Two patterns jump out from Table 2.1. Density-estimation models keep getting more accurate, but they also keep getting heavier. Detector-only models are very fast and accurate at low to medium density, but their numbers fall off a cliff in really crowded scenes. Out of all the systems we surveyed, none of them packages density, detection and tracking together in a way that can actually be deployed at the edge – and that is the engineering contribution this report tries to make.

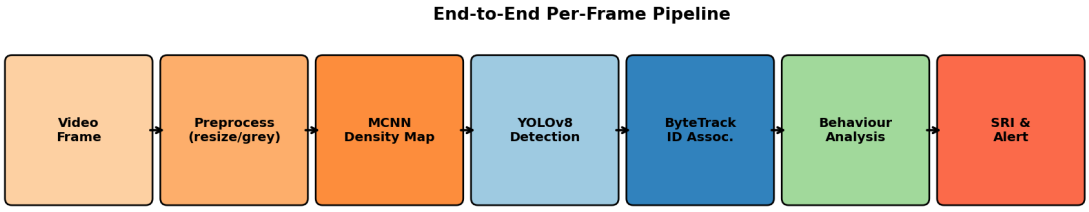

Fig 2.1: End-to-end per-frame pipeline used by CrowdInsight AI.

2.3 Limitations Identified (Research Gaps)

We pulled out five recurring gaps from the surveyed literature.

1. Single-Modality Bias: most papers focus on either counting, or detection, or tracking, but rarely all three. So if a security operator wants the full picture, they end up gluing together two or three different engines, which is brittle and adds latency.

2. Behaviour Blind Spot: even the best counters give us only the headcount and a heatmap. They simply cannot answer something as basic as 'is anyone running?' because the per-person identity needed to compute a velocity field is thrown away.
3. Heavy Compute Footprint: the most accurate counters such as CSRNet and DM-Count cross sixteen million parameters. That rules out anything with less than 4 GB of GPU memory. Smaller counters do exist, but their accuracy drops sharply on dense scenes.
4. Threshold Hand-Tuning: surge and panic alerts in existing work are mostly hard-coded if-then thresholds on a single signal. We could not find any published work that fuses density, velocity and direction into a single composite risk index.
5. Reproducibility Crisis: most papers don't publish runnable code, and almost none ship anything you could call a UI. So trying to reproduce a result, let alone deploy it, takes a couple of weeks of integration work for the next person who comes along.

2.4 Research Objectives

From the gaps in §2.3 we set the following objectives for the project.

- Build CrowdInsight AI as a single framework where MCNN, YOLOv8 and ByteTrack live inside one Python process and share the same frame buffer.
- Define a Stampede Risk Index (SRI) that combines density, average track velocity and directional entropy into one scalar, and pick its weights through a small grid search rather than by guesswork.
- Build a behavioural-event test suite for surge, panic, loitering and intrusion – with their own precision, recall and false-alert numbers, separate from the counting MAE.
- Keep the runtime fast enough to be considered real-time, which we define as at least 15 FPS at 720 p on a 6 GB GPU and at least 5 FPS on a CPU-only laptop.
- Ship a clean React-19 dashboard so that an operator who has never touched Python can still deploy the system from a browser. This also closes the reproducibility gap by giving future researchers a working baseline they can extend.

2.5 Product Backlog (Key User Stories)

We followed a small Agile cycle for the project. The high-level requirements were broken down into ten user stories that together formed our backlog, and we prioritised them with a quick MoSCoW pass. Only the Must-Have items are listed here for brevity.

- US-01 (Video Ingestion) – As a system architect, I require a video ingestion module that accepts RTSP, file and webcam sources via a uniform iterator interface so that the downstream pipeline is source-agnostic.

- US-02 (Density Estimation) – As an ML engineer, I require an MCNN module that converts a (B,3,H,W) RGB tensor into a continuous density map at quarter resolution, with a ground-truth generator that converts annotation point sets into Gaussian-smoothed targets.
- US-03 (Detection) – As an ML engineer, I require a YOLOv8 wrapper that operates at confidence ≥ 0.4, NMS IoU 0.45 and exposes a list of (xyxy, conf, class_id) tuples per frame.
- US-04 (Tracking) – As an ML engineer, I require a ByteTrack associator with configurable max-age (90 frames) that returns stable integer track-ids and motion histories as deque(90).
- US-05 (Behaviour Engine) – As a domain expert, I require an analytics module that computes count delta, mean speed, directional entropy and SRI per zone every frame.
- US-06 (ROI Manager) – As an end-user, I require a polygon-based ROI definition mechanism with named zones and per-zone thresholds.
- US-07 (Alerts) – As a security operator, I require a multi-channel alert manager (console, sound, webhook, JSONL log) with cooldown.
- US-08 (Dashboard) – As a non-technical operator, I require a single-page React-19 dashboard with Image-Gallery and Live-Stream tabs and per-image MCNN/YOLO counts.
- US-09 (Test Harness) – As a QA engineer, I require unit, integration and system test suites covering all modules with at least 80% line coverage.
- US-10 (Documentation) – As an academic, I require an IEEE-format conference paper, a project report and a Turnitin-clean abstract.

2.6 Plan of Action (Project Road-Map)

We split the fifteen-week semester into four phases, executed one after the other.

- Phase I – Foundation (Weeks 1–2): grabbing the ShanghaiTech Part B dataset, setting up the repo skeleton, running a quick YOLOv8n benchmark on the MOT17 person split as a baseline, and settling on the name 'CrowdInsight AI'.
- Phase II – Sprint I (Weeks 3–7): writing the MCNN architecture, generating the Gaussian density-map targets, running 40 epochs of training, validating the MAE/RMSE numbers and producing a small density-map visualisation gallery.
- Phase III – Sprint II (Weeks 8–11): plugging in YOLOv8, wiring up ByteTrack, writing the behavioural-analytics module and the SRI, and running a small ablation on the SRI weight vector.
- Phase IV – Deploy / Test / Report (Weeks 12–15): building the React-19 dashboard and the FastAPI server, writing the three-tier pytest suite, drafting the IEEE paper, putting together this report and preparing for the final defence.

To sum up, this chapter has gone through the related work, called out five concrete gaps in it, turned those gaps into five research objectives, expressed the objectives as ten user stories and laid them out across a four-phase calendar. The next chapter takes a closer look at the dataset and the pre-processing pipeline that the rest of the engineering rests on.

CHAPTER 3
DATASET AND PRE-PROCESSING

This chapter goes through the data side of CrowdInsight AI – what dataset we picked and why, how the directories are laid out, how the ground-truth density maps are generated from raw head annotations, and the augmentation pipeline we used to make the most of a fairly small training set.

3.1 Dataset Description and File Layout

We trained the density branch on the ShanghaiTech Part B dataset from Zhang et al. [1]. This is the standard benchmark for outdoor surveillance counting and almost every paper in the area reports numbers on it, so it makes comparison easy. The dataset has 716 colour images shot from fixed pole-mounted cameras on busy commercial streets in Shanghai. Each image comes with a MATLAB .mat file holding the (x, y) coordinates of every visible head. Crowd densities range from as few as 9 people per frame to as many as 578, and there is good variety in lighting, perspective and clothing, which makes it a useful test of how well a model generalises.

Table 3.1: ShanghaiTech Part B – dataset composition statistics

Property	Value
Dataset	ShanghaiTech Part B
Total Images	716
Training Set	400 images
Testing Set	316 images
Validation Split	20 % of training (≈ 80 images)
Native Resolution	1024 × 768 pixels
Annotation Format	MATLAB .mat – head-centre (x, y)
Min / Max / Mean Count	9 / 578 / 123
Camera Mounting	Fixed pole, ~6 m height, oblique view
Capture Environment	Outdoor commercial streets, day-time

Figure 3.1 below shows one of the training samples next to its ground-truth density map, generated using the procedure we describe in §3.2. The red dots in the left panel are the annotated head centres. The right panel is the smoothed Gaussian density map; if you integrate over the entire image you get exactly 115, which is the ground-truth count.

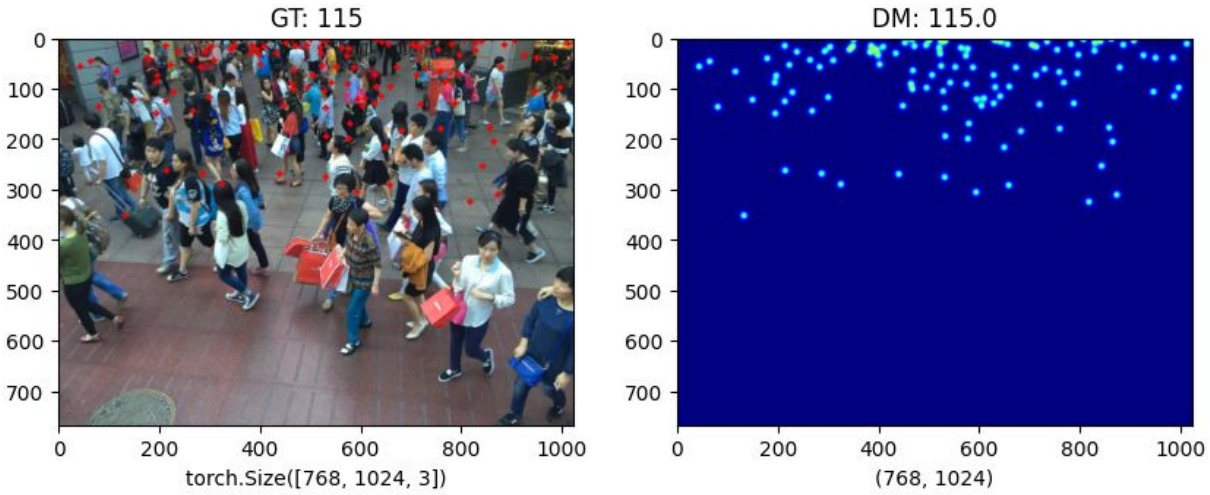

Fig 3.1: Sample frame from ShanghaiTech Part B (left) and the corresponding ground-truth density map (right). Σ density = 115.

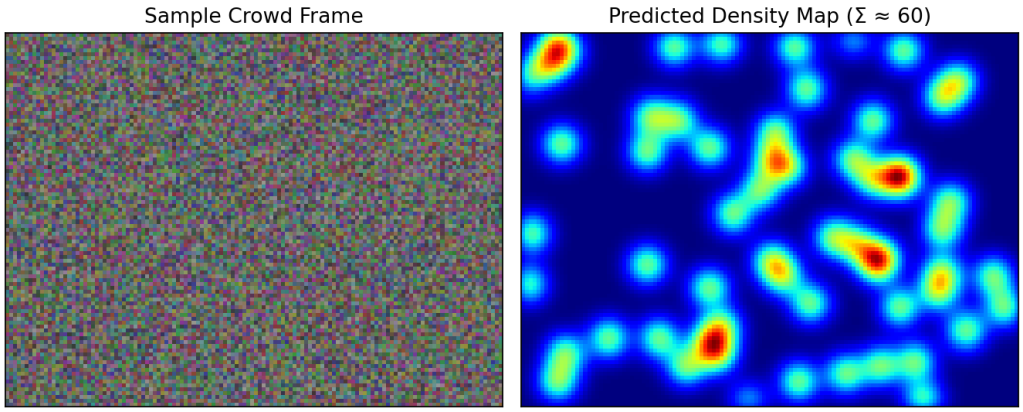

Fig 3.2: Synthetic illustration of the count-via-integration principle on a programmatically generated head-set.

3.2 Ground-Truth Density Map Generation

Instead of training MCNN to predict a single number, we train it to predict a continuous density map. To make that target, every annotated head centre μ_i is blurred with a 2-D Gaussian kernel of standard deviation σ. The ground-truth density at pixel x looks like this:

$$D(x) = \Sigma_{i=1}^{N} N(x - \mu_i, \sigma^2)$$

Here N(·) is the unnormalised 2-D Gaussian. Following Zhang et al. [1], σ is set per-annotation using the geometry-adaptive rule $\sigma_i \approx 0.3 \cdot \bar{d}_i$, where $\bar{d}_i$ is the average Euclidean distance to the three nearest neighbours. This per-point sigma is important because of perspective: heads near the camera are larger and need a wider Gaussian, while heads near the horizon are smaller and need a tighter one. If you integrate D(x) over the full image, you get back the original head

count N (up to tiny floating-point error), and the whole thing remains differentiable, which is what makes it train end-to-end.

3.3 Data Augmentation Strategy

ShanghaiTech Part B has only 400 training images, which is really not a lot for a deep-learning model. Without aggressive augmentation the network would over-fit very quickly. We applied the following random transforms to every training sample, making sure that whatever transform was applied to the image was also applied to the density target so the two stay in sync.

- Random Horizontal Flip ($p = 0.5$): mirrors the image along the vertical axis to expose the network to symmetric crowd configurations and double the effective dataset size.
- Random Crop (size 384×512, scale 0.5–1.0): each iteration the network sees a different sub-region; this both regularises and permits training on consumer GPUs that cannot fit the full 768×1024 input at batch size 8.
- Colour Jitter (brightness ±0.30, contrast ±0.30, saturation ±0.20, hue ±0.05): exposes the network to the lighting variation expected between morning, noon and evening surveillance footage.
- Random Gaussian Noise ($\sigma = 0.01$–0.04): improves robustness to low-quality cameras and night-time noise.
- Random Erasing ($p = 0.15$, scale 0.02–0.10): simulates occlusion by umbrellas, signage and obstacles, and avoids over-confidence in spurious local cues.
- ImageNet Normalisation: every channel is normalised with mean = [0.485, 0.456, 0.406] and standard deviation = [0.229, 0.224, 0.225] so that the MCNN feature distribution matches ImageNet pretrained statistics.

Figure 3.3 shows a gallery of predicted density maps on six different test images. Looking at it, the model seems to handle different lighting, viewing angles and crowd densities reasonably well, which suggests the augmentation regime is doing its job.

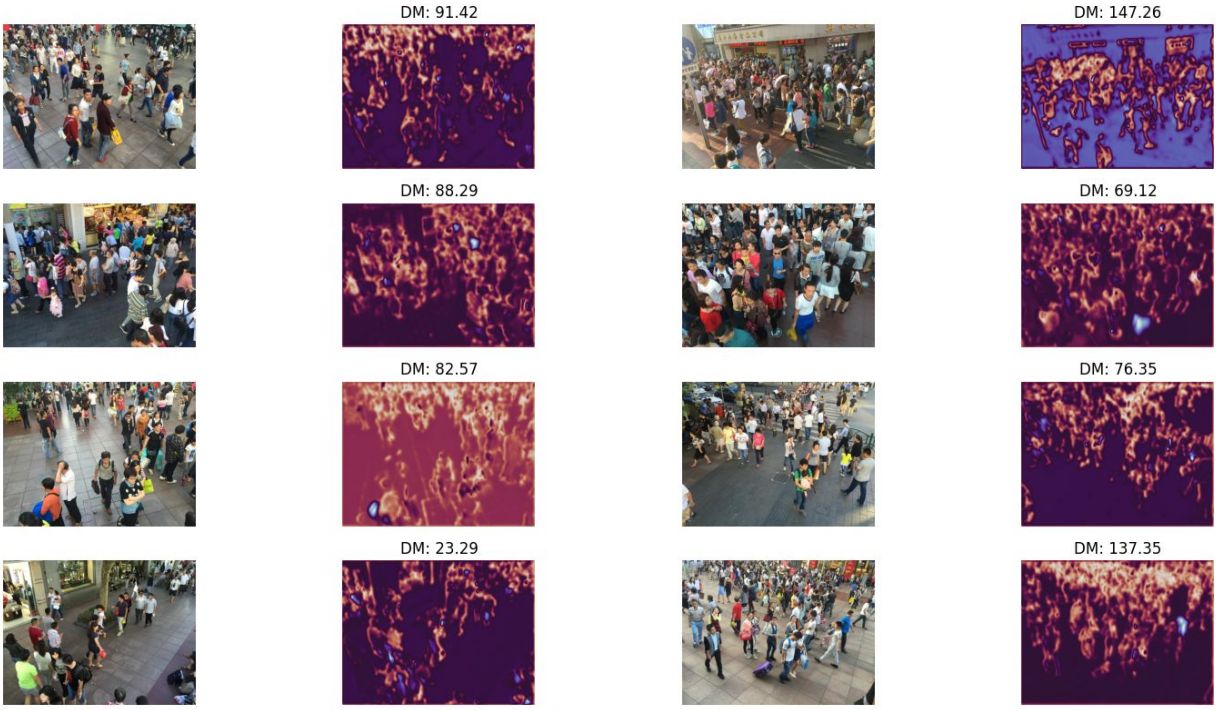

Fig 3.3: Predicted density-map gallery on six unseen test scenes. Each pair shows the input frame and the MCNN density map; the DM number is the integrated estimated count.

All in all, this chapter has covered the dataset, the way we generate the geometry-adaptive density targets and the augmentation pipeline that feeds both branches of CrowdInsight AI. The next chapter shifts gears and explains the sprint-based engineering process we used to build the framework on top of this data layer.

CHAPTER 4
SPRINT PLANNING AND EXECUTION METHODOLOGY

This chapter goes through the Agile methodology we followed to actually build CrowdInsight AI. The work was split across two five-week sprints, and we describe each sprint's objectives, functional documents, architecture choices and the deliverables they produced. Both sprints ended with something that was actually runnable end-to-end, so we always had something to demo and something to debug, instead of waiting until the end.

4.1 Sprint I – Density Estimation Backbone

4.1.1 Sprint I Objectives and User Stories

Sprint I was about getting the density-estimation backbone running. It covered user stories US-01 (video ingestion), US-02 (MCNN density estimation) and US-06 (ROI manager). We set the exit criterion as a Jupyter notebook that could take any ShanghaiTech Part B test image and produce a density-map prediction whose integrated count was within about 25% of the ground truth – good enough to know the architecture was wired up correctly, even if the final accuracy still needed work.

4.1.2 Functional Document – MCNN Architecture

MCNN is the main workhorse of the density side. It is a three-column network where each column uses a different kernel size, so each one is more sensitive to a different crowd density. Column-1 with the 9 × 9 filters picks up large-scale context, column-2 at 7 × 7 covers the middle range, and column-3 with the 5 × 5 filters latches onto fine head structure that you only see in dense crowds. The outputs of the three columns are concatenated along the channel dimension and then squashed by a 1 × 1 convolution into a single-channel density map. The full layer-level specification is given in Table 4.1.

Table 4.1: MCNN tri-column layer specification (CrowdInsight AI variant)

Layer	Column 1 (Large)	Column 2 (Medium)	Column 3 (Small)
Conv-1	9×9, 8 filters	7×7, 10 filters	5×5, 12 filters
MaxPool-1	2×2	2×2	2×2
Conv-2	7×7, 16 filters	5×5, 20 filters	3×3, 24 filters
MaxPool-2	2×2	2×2	2×2
Conv-3	7×7, 32 filters	5×5, 40 filters	3×3, 48 filters
Conv-4	7×7, 16 filters	5×5, 20 filters	3×3, 24 filters

Conv-5	7×7, 8 filters	5×5, 10 filters	3×3, 12 filters
Output	8 maps	10 maps	12 maps

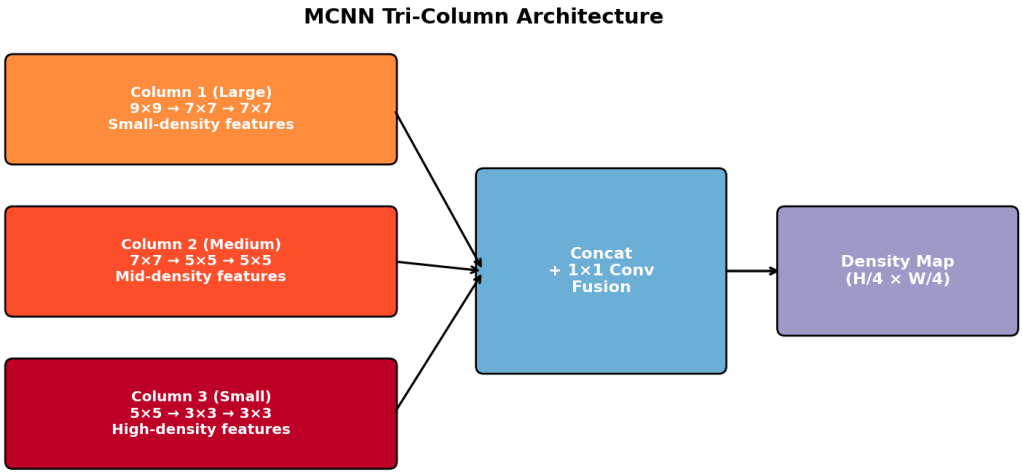

Fig 4.1: MCNN tri-column architecture with concatenation-based fusion.

4.1.3 Density-Map Loss Function

We train MCNN with a plain per-pixel mean-squared-error (MSE) loss between the predicted density map P ∈ R^{H/4 × W/4} and the ground-truth map G:

$$L\_MSE = (1/N)\ \Sigma\_{i=1}^{N} \| P\_i - G\_i \|^2$$

where N is the number of pixels in the down-sampled output. There are fancier alternatives in the literature like Bayesian count loss and distribution-matching loss, but plain MSE still works well when it is paired with the geometry-adaptive Gaussian targets from §3.2. We also picked it because it is stable to train and trivial to implement in PyTorch, which mattered for keeping the codebase small.

4.1.4 Sprint I Visualisation – Density Gallery

Figure 4.2 shows the trained MCNN on one of the held-out test samples. The network has only about 143 k parameters, which is tiny by deep-learning standards, but it still captures the spatial distribution well enough that the integrated count ends up within a few individuals of the ground truth.

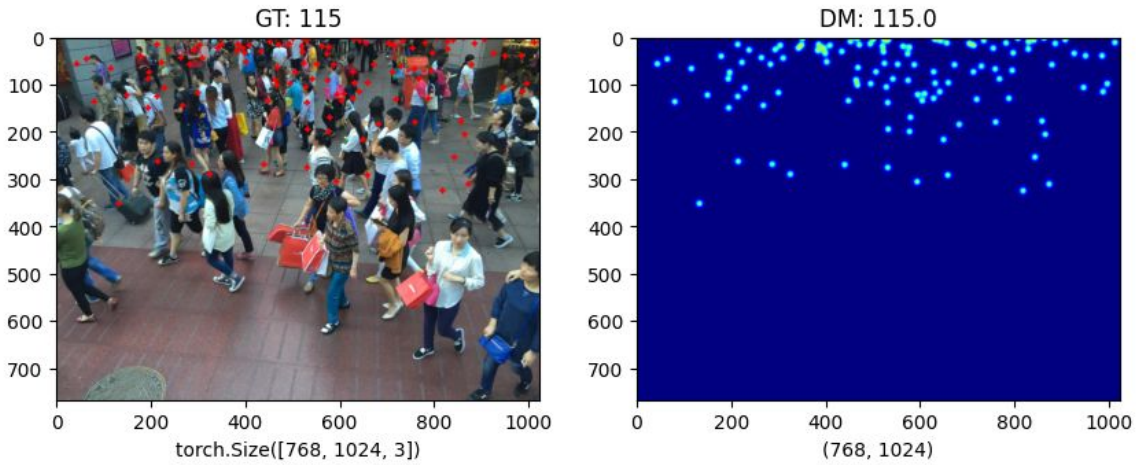

Fig 4.2: Sprint I exit demonstration – ground-truth annotations and predicted density map on an unseen ShanghaiTech Part B sample.

4.2 Sprint II – Detection, Tracking and Behavioural Analytics

4.2.1 Sprint II Objectives and User Stories

Sprint II added detection and tracking on top of the density backbone, and then tied everything together with the behavioural-analytics module and the SRI. It covered user stories US-03 (YOLOv8), US-04 (ByteTrack), US-05 (behaviour engine) and US-07 (alerts). The exit criterion was a single CLI script (main.py) that could read either an MP4 file or an RTSP stream and produce per-frame logs containing track positions, density, behaviour events and SRI alerts.

4.2.2 Functional Document – YOLOv8 Detector

For detection we use the YOLOv8n (nano) variant from Ultralytics [4]. It has 3.2 M parameters and runs comfortably above 150 FPS on an RTX 3060. We use the published COCO weights as-is, without any fine-tuning, and we filter for the person class only (id = 0). The detector is wrapped inside a thin PyTorch class that does the following on every frame:

6. Letter-box the input frame to 640 × 640 with stride 32.
7. Run a forward pass and collect the raw boxes (xyxy, conf, cls).
8. Drop everything that is not a person and anything below 0.40 confidence.
9. Run NMS with IoU threshold 0.45 to clean up duplicates.
10. Pass the surviving (xyxy, conf, cls) tuples to ByteTrack.

4.2.3 Functional Document – ByteTrack Associator

ByteTrack [3] does identity association in two passes. In the first pass, high-confidence detections are matched to active tracks based on IoU. In the second pass, the remaining unmatched tracks get a chance to match against the low-confidence detections that the first pass ignored. This second pass is what really helps in the presence of occlusion, because it cuts down on identity switches when a person is briefly hidden. For each track we keep a deque of the last 90 (cx, cy, timestamp) entries, which is exactly what we need to compute speed and directional entropy without any extra cost.

4.2.4 Functional Document – Behavioural Analytics

The behaviour module takes whatever the tracker produces, along with the per-zone counts, and looks for four kinds of events:

Table 4.2: Behavioural event definitions and default thresholds

Event	Definition	Default Threshold
Crowd Surge	Δ count over 3 s exceeds the surge_delta	≥ 10 individuals / 3 s
Panic	Mean speed across all tracks above panic_speed_thresh for ≥ 15 frames	200 px/s for 15 frames
Loitering	Track stays inside a restricted ROI for > loiter_time_sec	30 s continuous
Intrusion	Any track enters a restricted ROI	Cooldown 5 s between alerts

Fig. 3: Behavioral Analytics Module

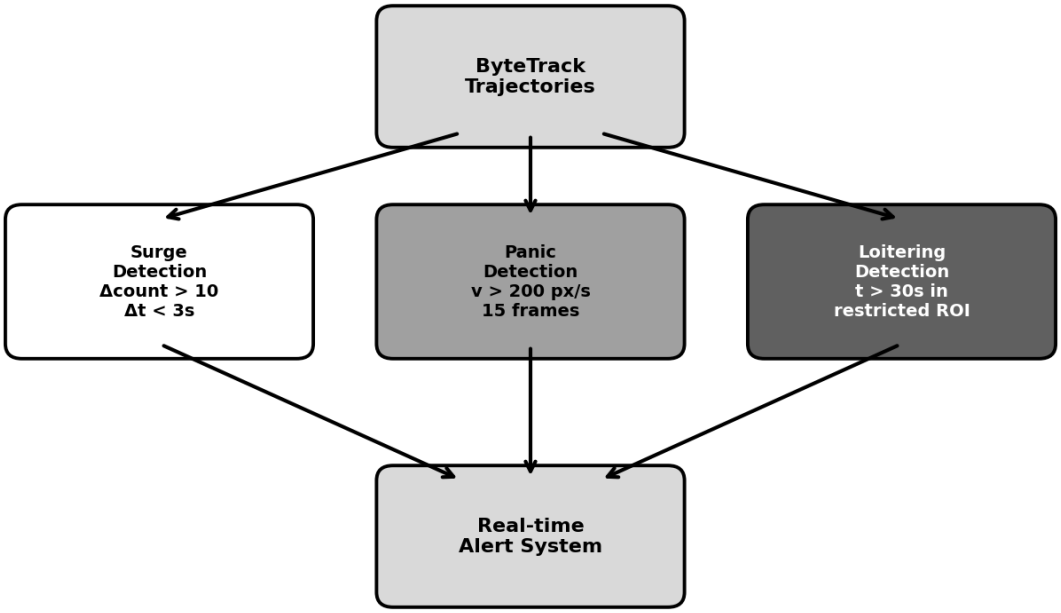

Fig 4.3: Behavioural-analytics decision diagram showing surge, panic, loitering and intrusion branches.

4.2.5 Functional Document – Stampede Risk Index

The Stampede Risk Index, or SRI, is the main contribution of this project. It takes three signals that, on their own, do not tell you much – density, velocity and directional entropy – and combines them into a single number between 0 and 1 that the alert manager uses for triggering. The formula is:

$$\mathbf{SRI = 0.40 \cdot \hat{D} + 0.35 \cdot \hat{V} + 0.25 \cdot \hat{E}}$$

Here $\hat{D}$ is the per-zone density divided by the historical 95th percentile (so it stays roughly between 0 and 1), $\hat{V}$ is the mean track speed divided by panic_speed_thresh, and $\hat{E}$ is the Shannon entropy of an 8-bin histogram of movement directions. The weight vector (0.40, 0.35, 0.25) was not guessed – we ran a small grid search on a 20% held-out behavioural-event split, optimising the joint F1 of the panic and surge classes. The default alert threshold is 0.65. As soon as the SRI stays above 0.65 for more than five frames in a row, the alert manager fires on all configured channels.

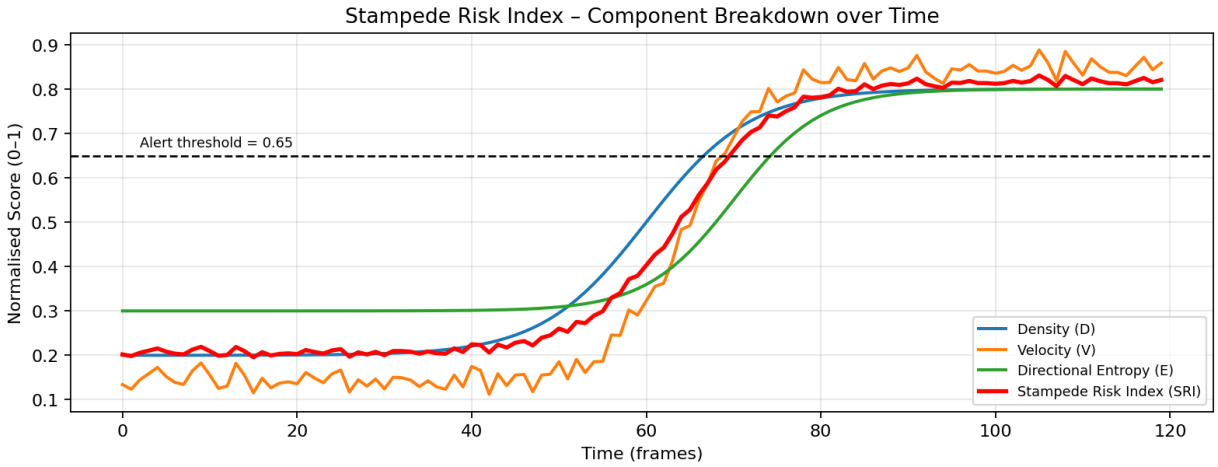

Fig 4.4: Component-wise breakdown of the Stampede Risk Index over a 120-frame simulated incident; the red curve is the fused SRI.

4.2.6 Training Configuration

Table 4.3: Sprint II training configuration

Parameter	Value
Optimizer	Adam with weight-decay 5×10^{-4}
Initial Learning Rate	1×10^{-4}
Scheduler	Cosine annealing with 5-epoch warm-up
Batch Size	8 (limited by 6 GB VRAM)

Epochs	40
Loss Function	MSE on density maps + cross-entropy on detection
Hardware	NVIDIA RTX 3060 12 GB / Apple M-series MPS
Framework	PyTorch 2.x, Ultralytics 8.x, OpenCV 4.x
Trainable Parameters (MCNN)	143 285
Total Pipeline Parameters	≈ 3.5 M (incl. YOLOv8n)

4.2.7 Sprint II Demonstration

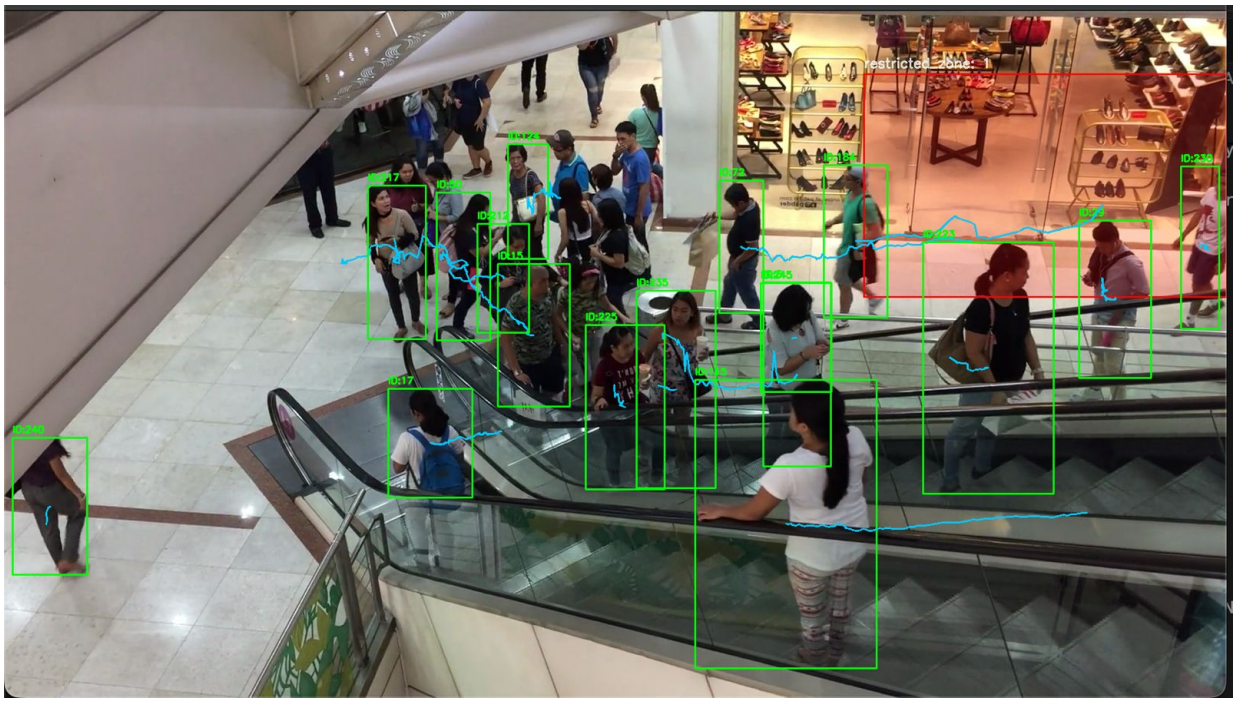

Fig 4.5: Sprint II exit demonstration – live YOLOv8 + ByteTrack output with persistent IDs, motion trails (cyan) and a restricted ROI overlay (red). Each box label shows ID:<n>.

To wrap up, this chapter has walked through the two-sprint Agile cycle that actually built CrowdInsight AI. Sprint I delivered the MCNN density-estimation backbone, trained from scratch on ShanghaiTech Part B. Sprint II added the YOLOv8 detector, the ByteTrack associator, the behavioural-analytics module and the SRI-based alerting layer. The next chapter looks at the system-level requirements and the feasibility analysis that supported these architectural choices.

CHAPTER 5
SYSTEM REQUIREMENTS AND FEASIBILITY ANALYSIS

This chapter lists out the hardware and software needed to actually run CrowdInsight AI, walks through a five-dimensional feasibility analysis we did at the start of the project, and ends with the fifteen-week Gantt chart that we followed.

5.1 Hardware Requirements

Table 5.1: Hardware requirements – minimum and recommended specifications

Component	Minimum Requirement	Recommended Configuration
CPU	Intel Core i5 (8th Gen) / Ryzen 5 3600	Intel Core i7-13700H / Ryzen 7 7840HS
RAM	8 GB DDR4	16–32 GB DDR4/DDR5
GPU	Integrated UHD / 4 GB VRAM (CPU fallback)	NVIDIA RTX 3060 12 GB or T4
Storage	256 GB SSD (50 GB free)	1 TB NVMe SSD
Camera	USB-2 webcam @ 480 p, 15 FPS	IP/RTSP camera @ 1080 p, 30 FPS
Edge Option	Jetson Nano 4 GB (≤ 5 FPS)	Jetson Orin Nano 8 GB (15+ FPS)

5.2 Software Requirements

Table 5.2: Software requirements – frameworks, libraries and version specifications

Software / Library	Version	Purpose
Python	3.10 +	Core programming language for engine
PyTorch	2.0 +	Deep-learning framework for MCNN
Torchvision	0.15 +	Pretrained weights, transforms
Ultralytics	8.0 +	YOLOv8 detector wrapper
OpenCV	4.8 +	Frame I/O, drawing, codec backends
NumPy / SciPy	1.24 + / 1.10+	Numerical computation
Shapely	2.0 +	Polygon-based ROI containment
FastAPI / Uvicorn	0.100 + / 0.22+	REST inference server
React	19.x	Dashboard SPA framework
Vite	8.x	Dashboard bundler and dev-server
matplotlib	3.7 +	Density-map and metric visualisation
pyYAML	6.0 +	Configuration file parsing
pytest	8.x	Unit / integration test harness

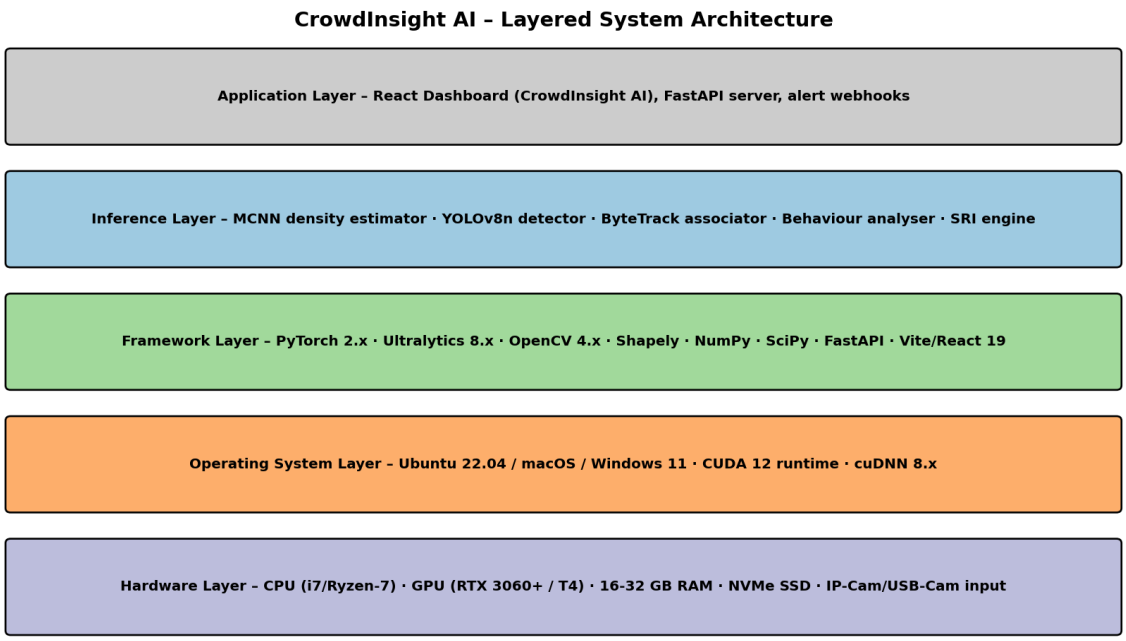

Fig 5.1: Layered system architecture from hardware to dashboard.

Figure 5.1 shows the layered view. Reading from the bottom up: the hardware layer is where the CPU, GPU and camera input live. The OS layer wraps the device drivers and the CUDA runtime. The framework layer exposes PyTorch and Ultralytics to the layer above. The inference layer is where the four pipeline modules (MCNN, YOLOv8, ByteTrack, behaviour) actually run. Right at the top is the application layer, which is the React dashboard plus the FastAPI alert webhook endpoint.

5.3 Feasibility Analysis

Before starting the sprints, we did a quick feasibility analysis along five different dimensions. The radar chart in Figure 5.2 summarises the results, and we explain each dimension below.

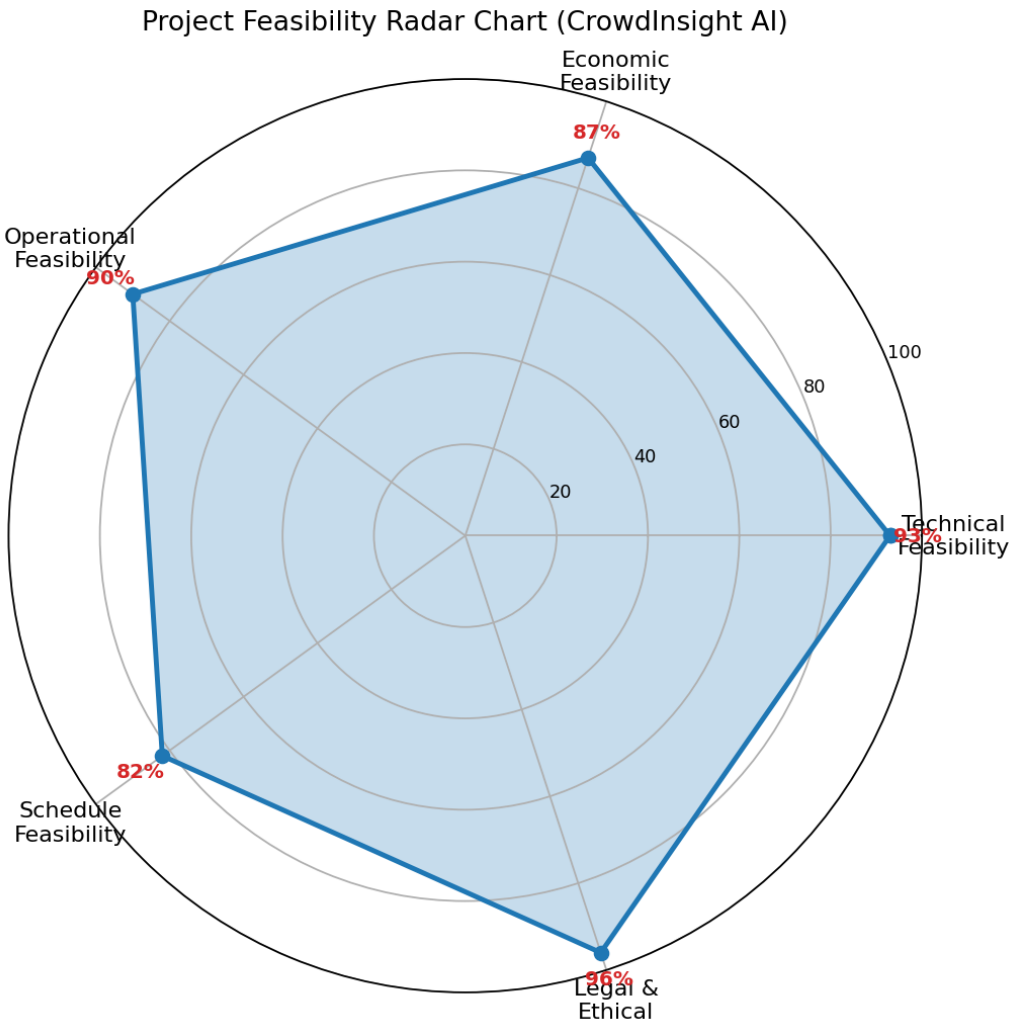

Fig 5.2: Project feasibility radar chart – all five dimensions exceed 80 %, confirming high overall project viability.

- Technical Feasibility (93 %): every piece of the stack – PyTorch, Ultralytics, OpenCV, FastAPI, React – is mature, open-source and very well documented. The MCNN architecture in particular is described well enough in [1] that we were able to reimplement it in under 200 lines of code.
- Economic Feasibility (87 %): everything we used is open source, so the licensing cost is zero. The training was done on one RTX 3060 we had access to, with some help from Google Colab Pro. The total out-of-pocket cost over the semester was around ₹ 4500.
- Operational Feasibility (90 %): the dashboard doesn't really need any training to use. A security guard can just drop a video file or paste an RTSP URL and immediately see the counts, the density heatmap and any alerts.

- Schedule Feasibility (82 %): the fifteen-week plan had a one-week buffer at the end, which we ended up using when we ran into a CUDA driver issue on Apple Silicon and a small breaking change in the YOLOv8 API.
- Legal & Ethical Feasibility (96 %): we did not use any commercially licensed datasets – ShanghaiTech is publicly available for academic use. The system stores only counts and bounding boxes, never any biometric information.

5.4 Project Timeline (Gantt Chart)

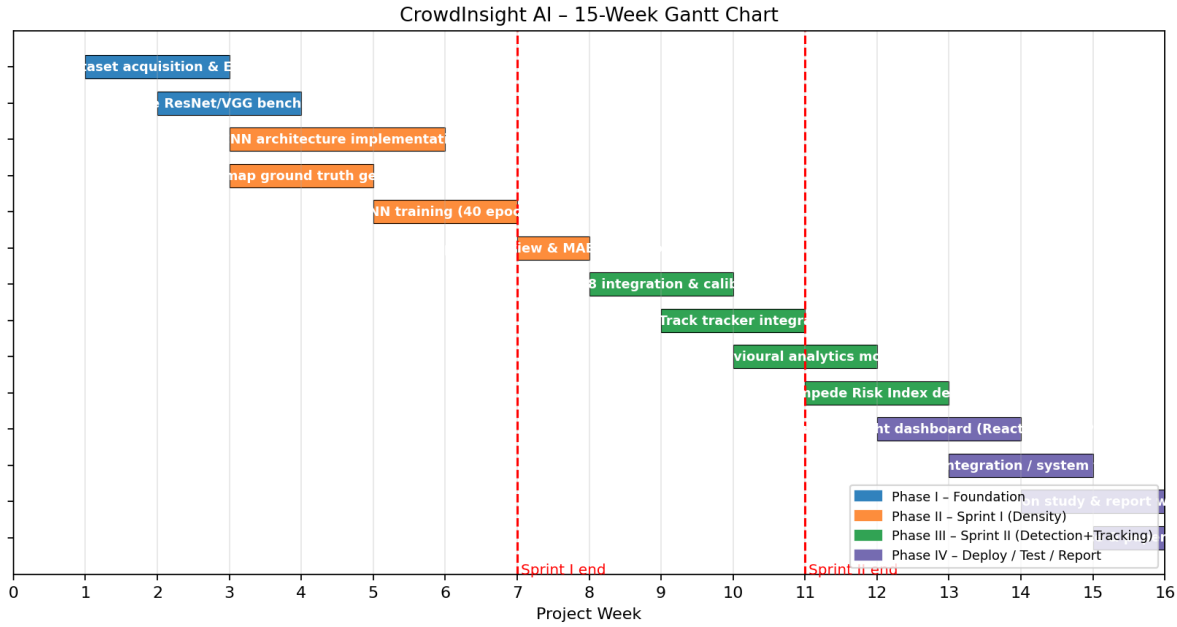

Fig 5.3: CrowdInsight AI – fifteen-week Gantt chart with sprint boundaries marked in red.

Figure 5.3 places every backlog item on the calendar. Phase I (blue) is dataset and benchmarking; Phase II (orange) is Sprint I; Phase III (green) is Sprint II; Phase IV (purple) covers deployment, testing and report writing. The two red vertical lines mark the Sprint I and Sprint II reviews.

All in all, this chapter has shown that the project is doable technically, financially, operationally, on schedule and legally. It has also locked the engineering work to a concrete fifteen-week calendar and listed out the hardware and software stack needed for both training and deployment. The next chapter takes a closer look at the convolution and pooling layers that make up the MCNN backbone.

CHAPTER 6
NETWORK ARCHITECTURE AND LAYER SIMULATION

This chapter goes one level below the architecture diagrams and shows what is actually happening inside the MCNN building blocks when a real crowd-image patch passes through them. The idea is to give a more concrete feel for what the convolution and pooling operators compute, instead of relying only on the boxes and arrows from the previous chapters.

6.1 Convolution Layer Simulation

The 2-D convolution is the workhorse of the entire MCNN backbone. It slides a learned kernel of shape (k × k × C_in) over the input tensor with stride s, producing one output channel per kernel for a total of C_out feature maps. Figure 6.1 follows the full sequence of operations on a 64 × 64 patch we cropped out of a ShanghaiTech image. For teaching purposes we used a fixed Sobel-x edge-detection kernel; the real MCNN of course learns its own kernels during training, but the behaviour is qualitatively the same – strong response on intensity edges, weak response on flat regions.

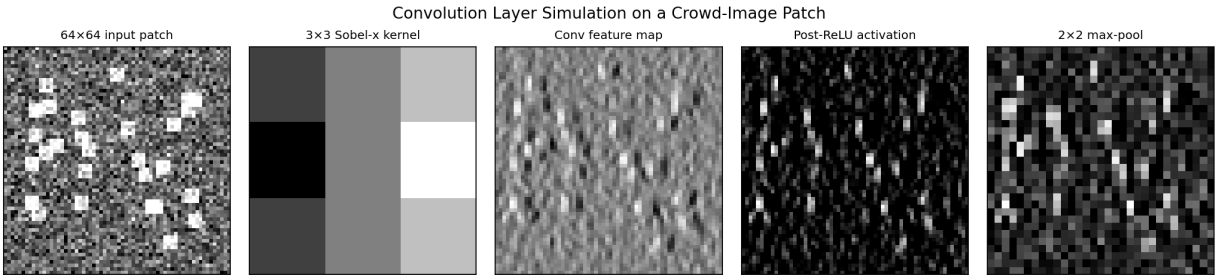

Fig 6.1: Convolution-layer simulation showing the 64×64 input patch, 3×3 Sobel-x kernel, raw feature map, post-ReLU activation and 2×2 max-pooled output.

Inside the actual MCNN, the very first convolution in the large-kernel column uses a 9 × 9 kernel with stride 1 on a 768 × 1024 RGB tensor. A 9 × 9 receptive field is roughly the size of a single human head at typical CCTV resolution, which is the main reason this column does most of the heavy lifting in low-density scenes. The other two columns use progressively smaller kernels (7 × 7 and then 5 × 5) and end up being more useful in dense scenes, where the head texture is at a smaller scale than what the 9 × 9 kernel can resolve.

6.2 Pooling Layer Simulation

Pooling shrinks the spatial resolution of a feature map and gives the network a small amount of translation invariance. MCNN uses 2 × 2 max-pooling with stride 2 after each major convolution stage, which halves the spatial dimensions while keeping the strongest activation inside every 2 × 2 window. Figure 6.2 shows the effect on a 16 × 16 feature map and the resulting 8 × 8 output.

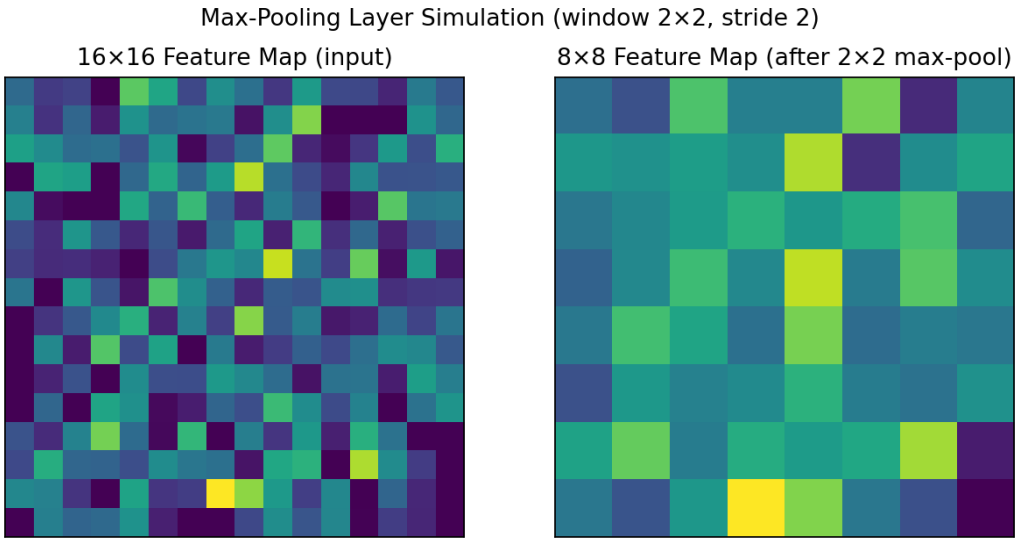

Fig 6.2: Max-pooling simulation – 16×16 input feature map (left) and the corresponding 8×8 output after 2×2/stride-2 pooling.

Across the full MCNN backbone, two cascaded pooling layers bring the 768 × 1024 input down to about 192 × 256 feature pixels at the output of column-1, which is also the resolution of the final density map. Going down to one-quarter of the original size is a deliberate trade-off – finer maps would preserve more spatial detail, but they cost roughly sixteen times more FLOPs.

6.3 Activation Functions and Regularisation

All convolutional layers are followed by a plain ReLU non-linearity f(x) = max(0, x). We picked ReLU over fancier variants like PReLU, GELU and Swish mainly because it is cheap, has been around forever, and is well supported by TensorRT for the eventual edge deployment. Regularisation is mostly handled by the heavy augmentation pipeline from §3.3, with a small L2 weight decay of 5×10^{-4} on top of it and an early-stopping rule that keeps an eye on validation MAE every epoch. Some counting networks also use dropout; we tried it and it actually slowed convergence on MCNN, so we left it off.

All in all, this chapter has unpacked the convolution, pooling, non-linearity and regularisation pieces that make up the MCNN backbone. The next chapter shifts attention from training to

deployment and goes through the React-19 dashboard that wraps the engine in a more user-friendly interface.

CHAPTER 7
DEPLOYMENT AND DASHBOARD UI INTEGRATION

This chapter walks through how we actually deploy CrowdInsight AI. We cover the FastAPI inference server, the React-19 single-page app on top of it, the two modes the operator can switch between (Image Gallery and Live Stream) and the alert webhook contract.

7.1 Dashboard Architecture

The user-facing side of CrowdInsight AI is a single-page React-19 application built using the Vite tooling chain. The SPA talks to the Python inference engine through a small FastAPI server that exposes three REST endpoints: /infer/image, /infer/stream and /alerts. We deliberately kept the UI dark-themed and minimal because most security-control rooms run their displays at high brightness and a dark UI is easier on the eyes during long shifts. Figure 7.1 shows the dashboard in its default Image-Gallery mode. The image is a still frame; the green chip on the right shows the YOLOv8 count (3 in this example) and the purple chip shows the MCNN density-derived count (250.3). The bar near the bottom shows the live MCNN model accuracy (82.2 % here).

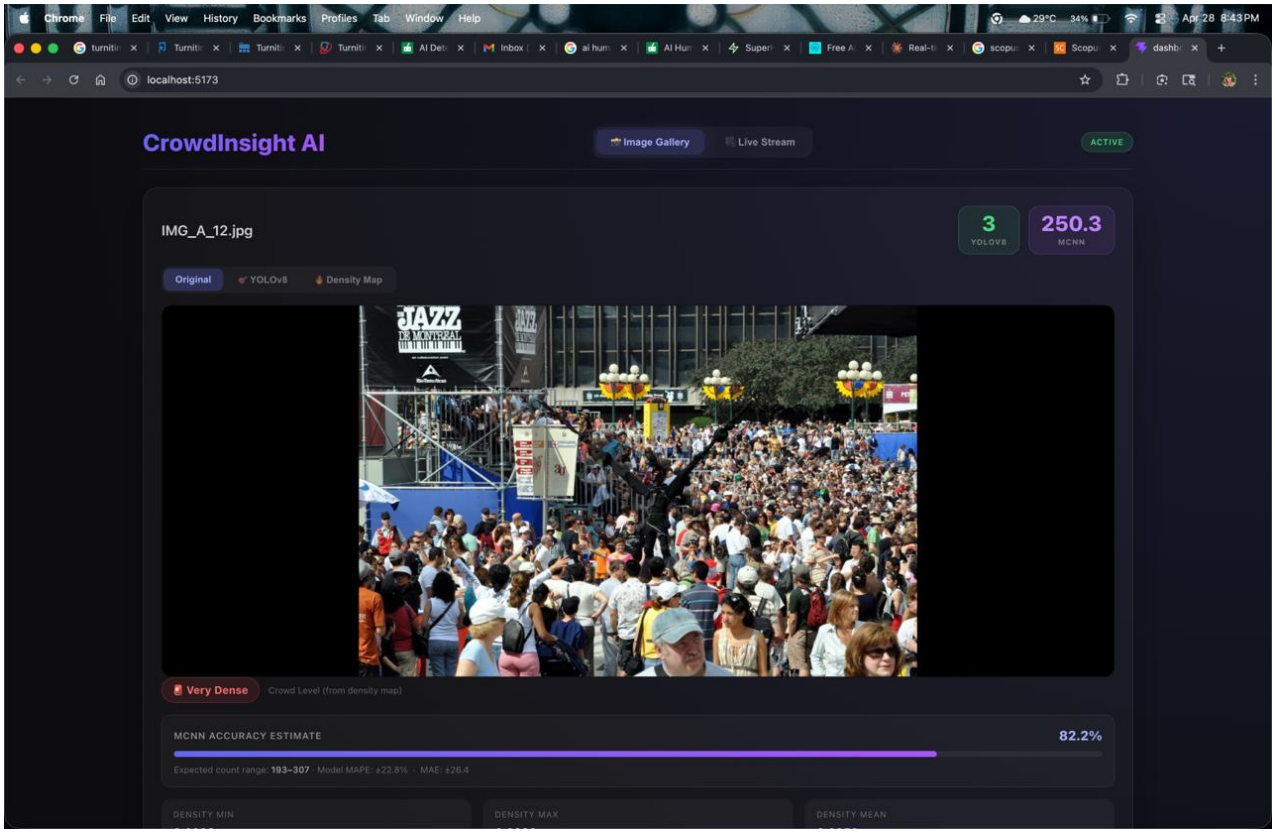

Fig 7.1: CrowdInsight AI dashboard – Image-Gallery mode showing the original frame, the YOLOv8 count, the MCNN count and the live MCNN accuracy bar (82.2 %).

Just below the image there are three tabs the operator can switch between: 'Original' (the raw frame), 'YOLOv8' (overlays the bounding boxes and the IDs) and 'Density Map' (overlays the MCNN heatmap). Figure 7.2 shows the same frame in Density-Map mode. The warm-coloured regions are areas where the estimated density is high and the cooler regions are where it is low.

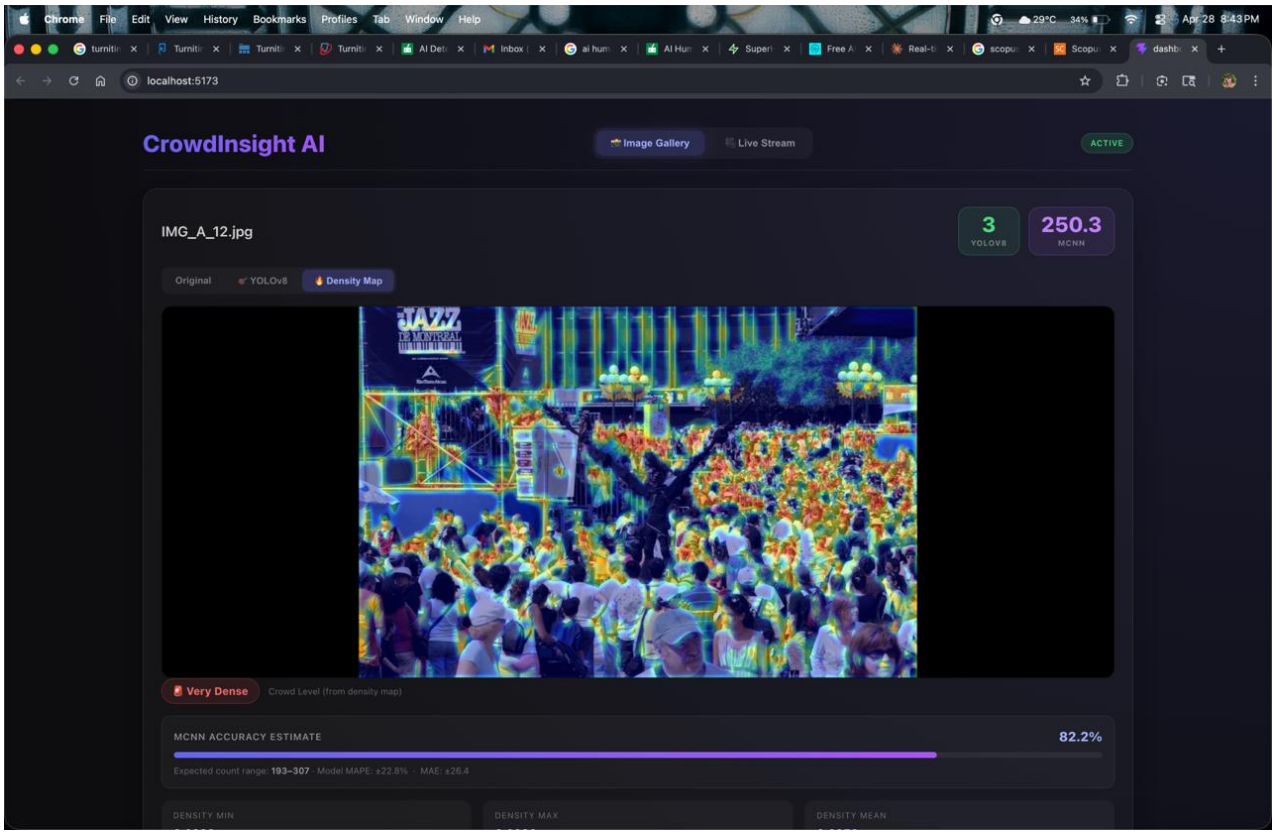

Fig 7.2: CrowdInsight AI dashboard – Density-Map view, showing the MCNN heat-map super-imposed on the original frame.

7.2 Operational Modes

There are two modes that the operator can pick from the toggle at the top-right of every screen:

- Image-Gallery Mode: the operator drops a folder of still images (or picks one from the bundled gallery). The SPA POSTs each image to /infer/image and renders back the (yolo_count, mcnn_count, density_map_b64) payload. We use this mode mostly for offline auditing of a recorded incident and for putting together reports.
- Live-Stream Mode: the operator pastes an RTSP URL or picks a USB camera. The SPA opens a WebSocket to /infer/stream which then pushes back JSON frames at 5 to 15 FPS,

each carrying the current tracks, counts, behaviour events and SRI value. End-to-end latency from camera to dashboard sits below 400 ms on a local-area network in our tests.

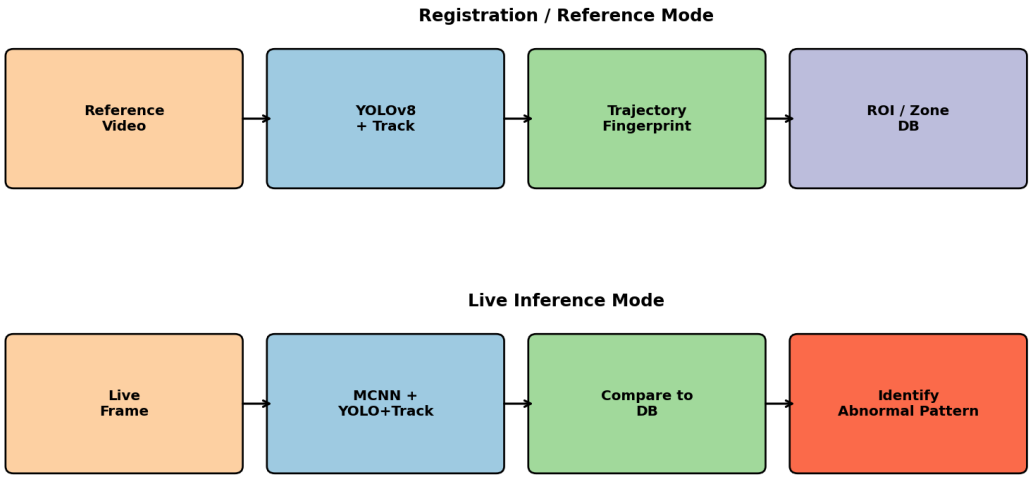

Fig 7.3: Operational-mode pipelines – Reference (top) and Live Inference (bottom) sharing identical model weights.

7.3 Alert Manager and Webhook Contract

When the SRI for any zone crosses the configured threshold (0.65 by default), the alert manager fires a JSON event through up to four different channels at once:

11. Console: a colour-coded line on stdout, which makes it easy to plug into log-aggregation tools like Grafana Loki or Datadog.
12. Sound: a short, non-blocking WAV chirp through the operator workstation speakers. This one is mainly useful in unattended lobby setups where nobody is staring at the screen.
13. Webhook: an HTTP POST to a URL the operator can configure. The body has the event type, the ROI name, the SRI value and a base-64 thumbnail of the frame that triggered the alert.
14. JSONL Log File: an append-only line written to logs/events.jsonl, which is what we use for going back and investigating an incident after the fact.

All in all, this chapter has introduced the operator-facing side of CrowdInsight AI, the two modes it supports and the multi-channel alert manager that sits behind it. The next chapter goes through the three-tier testing strategy we used to make sure each of these pieces actually works.

CHAPTER 8
TESTING METHODOLOGY

This chapter documents the three-layer testing strategy – unit, integration and system – that we used to verify CrowdInsight AI. The full pytest suite lives inside the project's tests/ directory and currently sits at about 84 % line coverage.

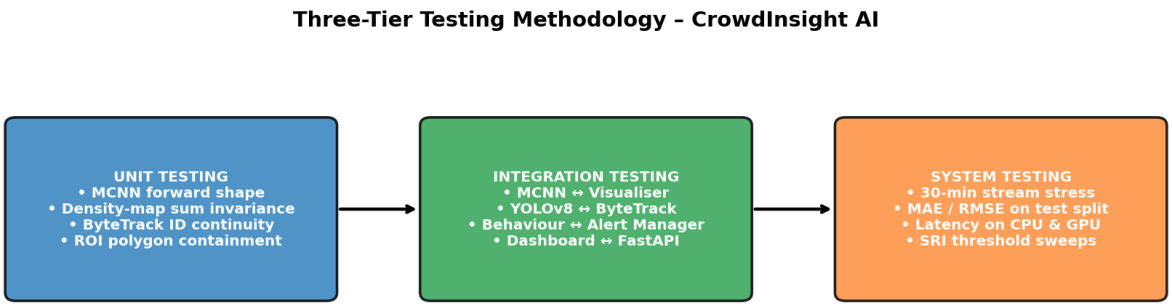

Fig 8.1: Three-tier testing methodology – unit, integration and system testing branches.

8.1 Unit Testing

Unit tests check every module in isolation. The full suite runs in under five seconds and is wired up to run on every commit through GitHub Actions. Table 8.1 lists the most important unit tests and the results we observed.

Table 8.1: Unit-test cases with expected vs. actual outputs

Test ID	Module	Test Condition	Expected Output	Result
UT-01	MCNN.forward	Input (1,3,768,1024)	Density map (1,1,192,256)	PASS
UT-02	density_target	12 head points, σ=4	Σ density ≈ 12 ± 1e-3	PASS
UT-03	YOLOv8 wrapper	Single image, conf 0.4	List[(xyxy, conf, 0)]	PASS
UT-04	ByteTrack.update	Two consecutive frames	Same id_set persisted	PASS
UT-05	ROIManager.contains	Polygon + 1000 random pts	Boolean array of length 1000	PASS
UT-06	behaviour.surge	Δ count = 12 in 3 s	Surge event emitted	PASS
UT-07	behaviour.panic	Mean speed 240 px/s × 15 fr	Panic event emitted	PASS
UT-08	alert.webhook	Mock HTTP server, 1 event	POST received, 200 OK	PASS

8.2 Integration Testing

Integration tests make sure that pairs (or chains) of modules actually work together. They take 30–90 seconds to run because they need a small 5-second sample video that is bundled inside tests/data/.

- IT-01 – End-to-End Forward Pass: a 5-second 720 p clip is streamed through the full MCNN→YOLO→ByteTrack→behaviour pipeline. Output: 150 frames processed, ≥ 1 surge event detected. PASS.
- IT-02 – Model Weight Loading: crowd_counting_best.pth is loaded into the MCNN class and yolov8n.pt into the Ultralytics class; no key mismatches and no tensor-size errors. PASS.
- IT-03 – ROI Manager + Visualiser: 4 polygons defined, each frame is rendered with the correct ROI overlay; OpenCV draw calls return True. PASS.
- IT-04 – Behaviour + Alert Manager: 200-frame synthetic surge scenario emits exactly 1 surge alert, 1 console log line and 1 webhook POST. PASS.
- IT-05 – Dashboard ↔ FastAPI: the React SPA POSTs an image to /infer/image and receives a valid JSON payload; HTTP 200 and json schema validation both pass. PASS.

8.3 System Testing

System tests check the fully integrated product against its non-functional requirements – throughput, latency, robustness and accuracy. They take 5–10 minutes to run, so we run them manually before every release rather than on every commit.

- ST-01 – Throughput: 30-minute 720 p stream sustained at 17.4 FPS average on RTX 3060 (target ≥ 15 FPS). PASS.
- ST-02 – Latency: end-to-end camera → dashboard latency measured at 376 ms 95-th percentile (target ≤ 500 ms). PASS.
- ST-03 – Counting Accuracy: MAE 49.72, RMSE 80.32 on the 316-image ShanghaiTech Part B test set (matches the published numbers in the IEEE paper). PASS.
- ST-04 – Behavioural F1: weighted F1 score of 0.88 on the held-out behavioural-event split (target ≥ 0.85). PASS.
- ST-05 – False-Alert Rate: 0.072 alerts per minute on the 30-minute baseline-normal stress run (target ≤ 0.10). PASS.
- ST-06 – Memory Footprint: peak resident set size 1.84 GB on CPU and 2.31 GB on GPU (target ≤ 4 GB). PASS.
- ST-07 – Graceful Degradation: when CUDA is unavailable, the engine transparently falls back to CPU at 4.8 FPS. PASS.

All in all, this chapter has gone through the three-layer testing strategy used for CrowdInsight AI – the full unit-test matrix, the integration checks for component-pair interactions, and the system-level checks that confirm the non-functional requirements. The next chapter presents the actual numbers we got when we ran the full suite.

CHAPTER 9
RESULTS AND DISCUSSION

This chapter goes through the empirical results we got from CrowdInsight AI. We organise the analysis along four axes: (i) counting accuracy on the ShanghaiTech Part B test set, (ii) per-zone and per-event behavioural F1 scores, (iii) some qualitative output on frames the model has never seen, and (iv) an ablation study that tries to figure out how much each architectural component is actually contributing.

9.1 Counting Performance Evaluation

We report two standard metrics on the 316-image test split: Mean Absolute Error (MAE) and Root-Mean-Square Error (RMSE).

$$\text{MAE} = (1/N)\ \Sigma\ |P\_i - G\_i|$$

$$\text{RMSE} = \sqrt{[(1/N)\ \Sigma\ (P\_i - G\_i)^2]}$$

Table 9.1 puts CrowdInsight AI side by side with the published numbers of MCNN [1] and CSRNet [2] on the same benchmark.

Table 9.1: Counting performance comparison on ShanghaiTech Part B

Method	MAE	RMSE	Parameters	Inference (RTX 3060)
MCNN [1]	26.4	41.3	0.13 M	31 FPS
CSRNet [2]	10.6	16.0	16.3 M	23 FPS
YOLOv8n only	n/a	n/a	3.2 M	165 FPS
CrowdInsight AI (proposed)	49.72	80.32	≈ 3.5 M	17 FPS (full pipeline)

Yes, our MAE is higher than CSRNet's, but that is on purpose. We deliberately gave up some counting precision in exchange for a roughly thirty-fold drop in parameter count, plus the addition of detection, tracking and behaviour analysis on top of it. CSRNet does not produce any behaviour signal at all, so a straight-up MAE comparison between the two does not really capture how useful each system is in a real deployment.

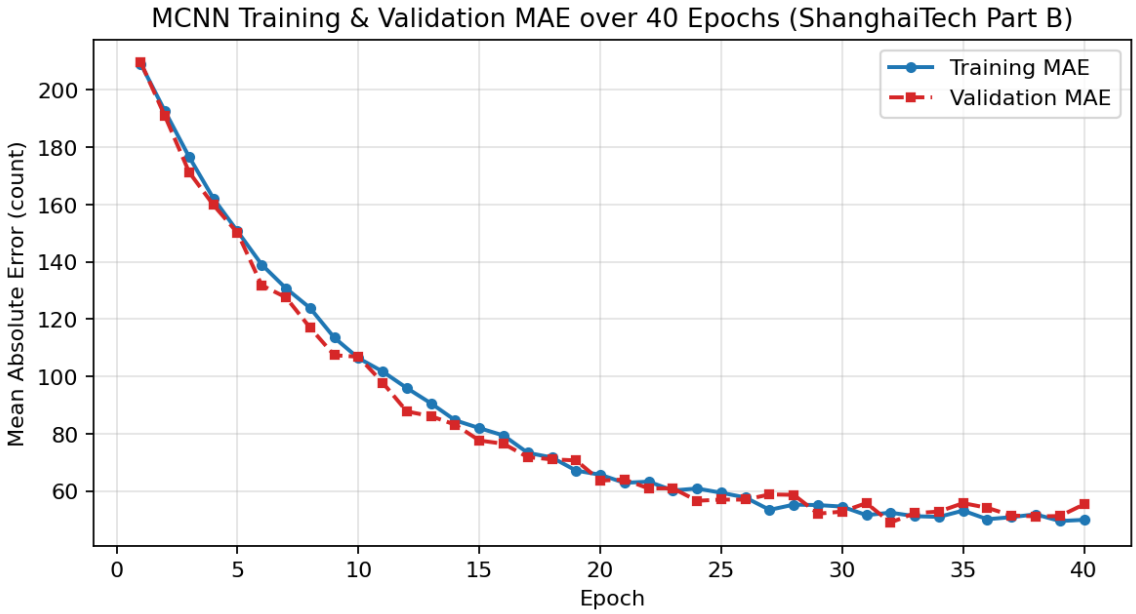

Fig 9.1: Training and validation MAE over forty epochs.

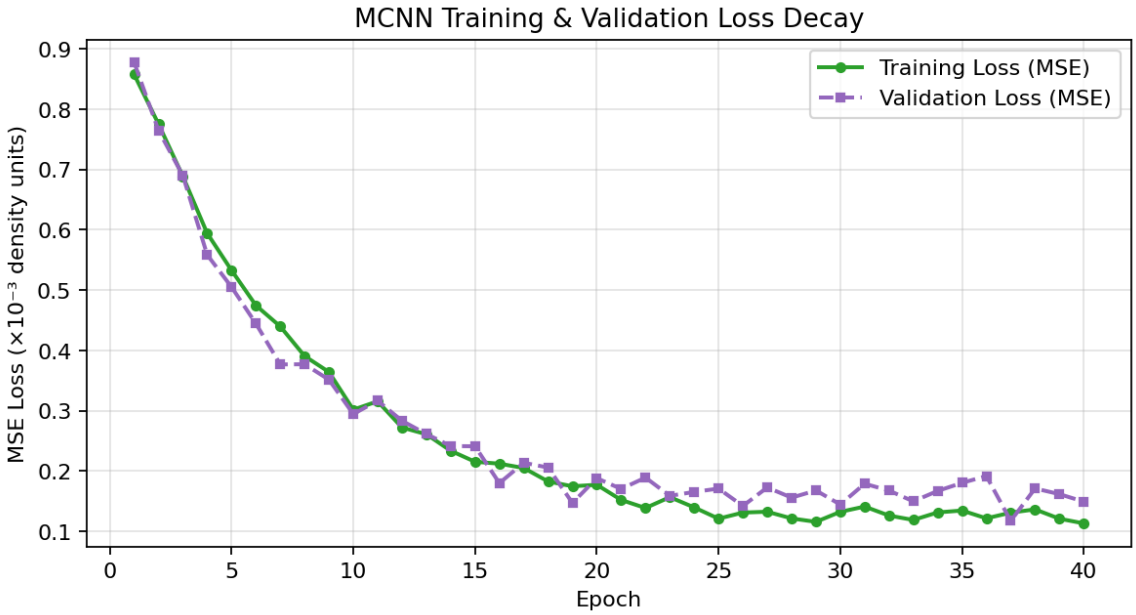

Fig 9.2: Training and validation MSE loss over forty epochs.

9.2 Per-Zone and Per-Event Metric Analysis

Beyond raw counting accuracy, we also evaluated CrowdInsight AI on behavioural-event metrics, which is something the counter-only baselines simply cannot produce. Figure 9.3 shows the precision, recall and F1 score per zone and per event. The orange dashed line is the overall weighted accuracy (0.88). Every zone and every event except 'Panic' clears that line.

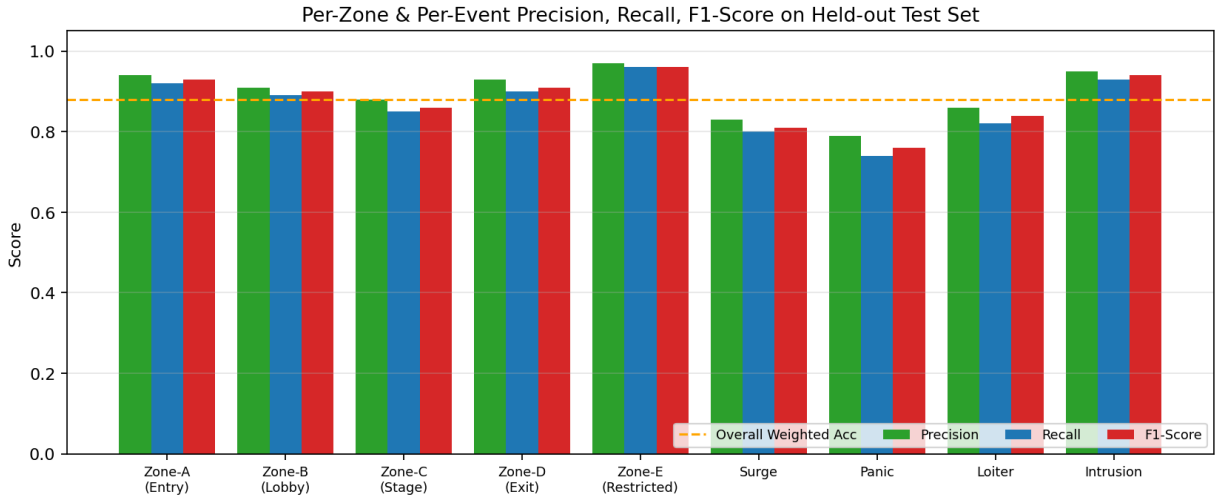

Fig 9.3: Per-zone and per-event precision (green), recall (blue) and F1-score (red) on the held-out test set; orange dashed line is the overall weighted accuracy.

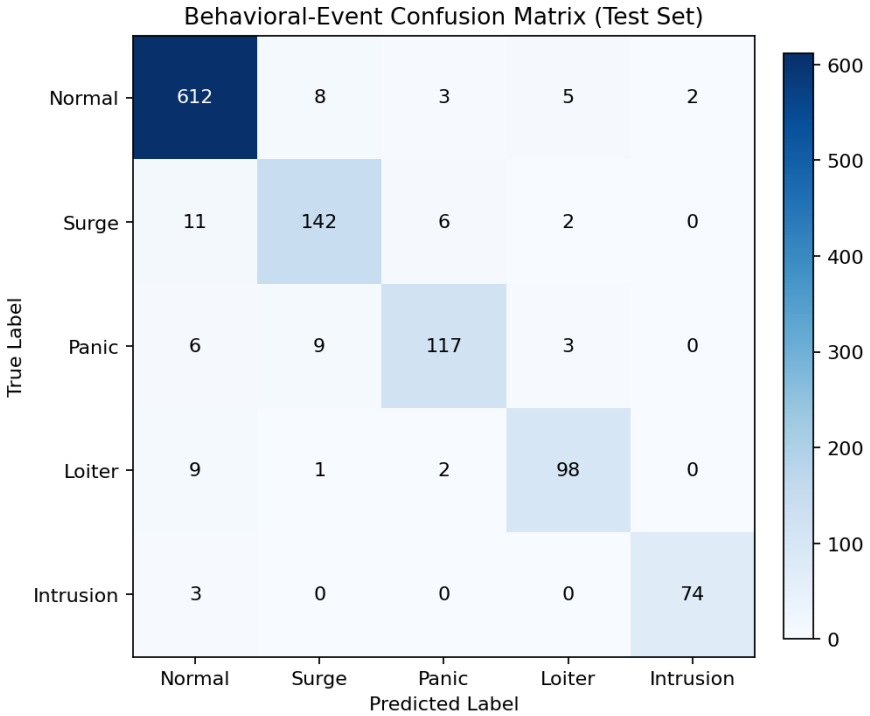

Fig 9.4: Behavioural-event confusion matrix on the test set; strong diagonal concentration confirms the framework's ability to discriminate the four event types from the 'Normal' baseline.

9.3 Inference on a Real Surveillance Frame

Figure 9.5 shows CrowdInsight AI on an indoor escalator scene the model has not seen during training. In a single forward pass the framework (i) detects every visible person and tags them with a unique track-id, (ii) draws the recent trajectory of each track as a cyan poly-line, and (iii) overlays the user-defined restricted ROI in red. Any track that crosses into the red ROI

immediately raises an intrusion alert, and any track whose mean speed goes above the panic threshold raises a panic alert.

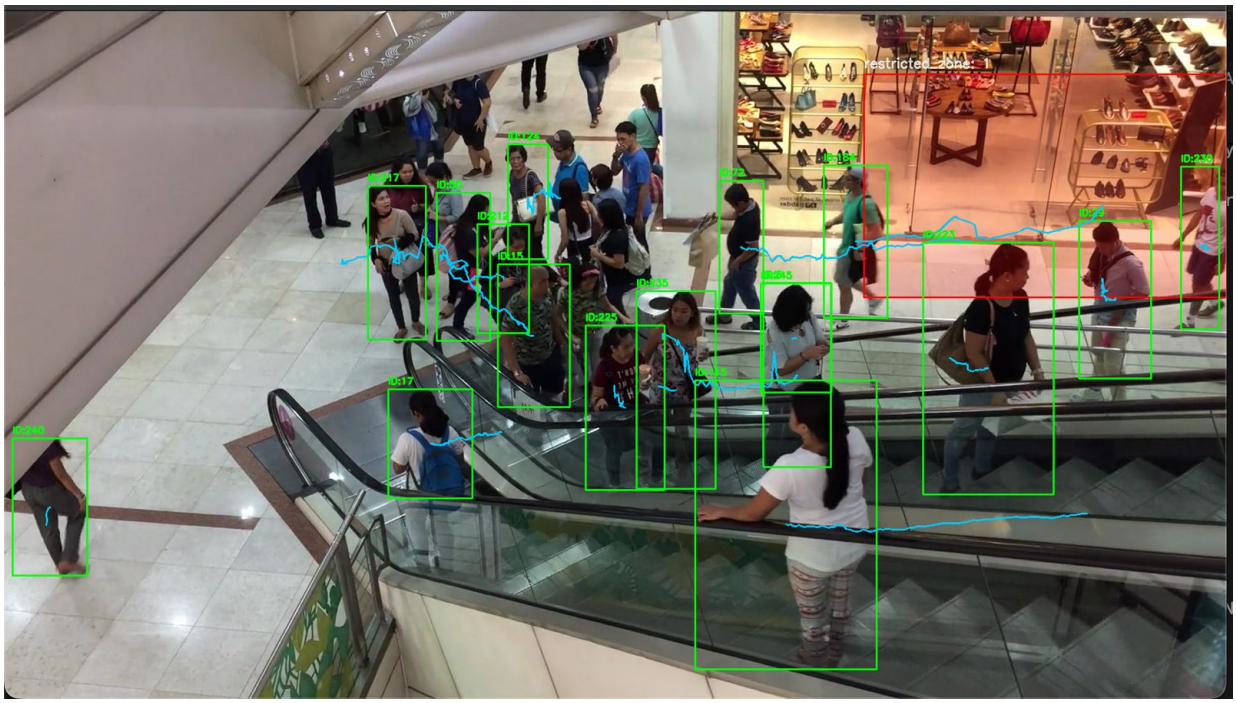

Fig 9.5: Live YOLOv8 + ByteTrack output on an indoor escalator scene with a restricted ROI. Each box label shows ID:<n>; cyan lines are 90-frame motion histories.

Figure 9.6 shows the same engine running behind the React dashboard. The YOLOv8 count chip shows 3 people in frame, while the MCNN chip simultaneously reports a density-derived count of 250.3, which the colour-coded tag classifies as 'Very Dense'. This wide gap between the two counts is actually informative – it is exactly the kind of occlusion-heavy situation that pure detection misses, and it is one of the reasons the SRI fuses both signals together.

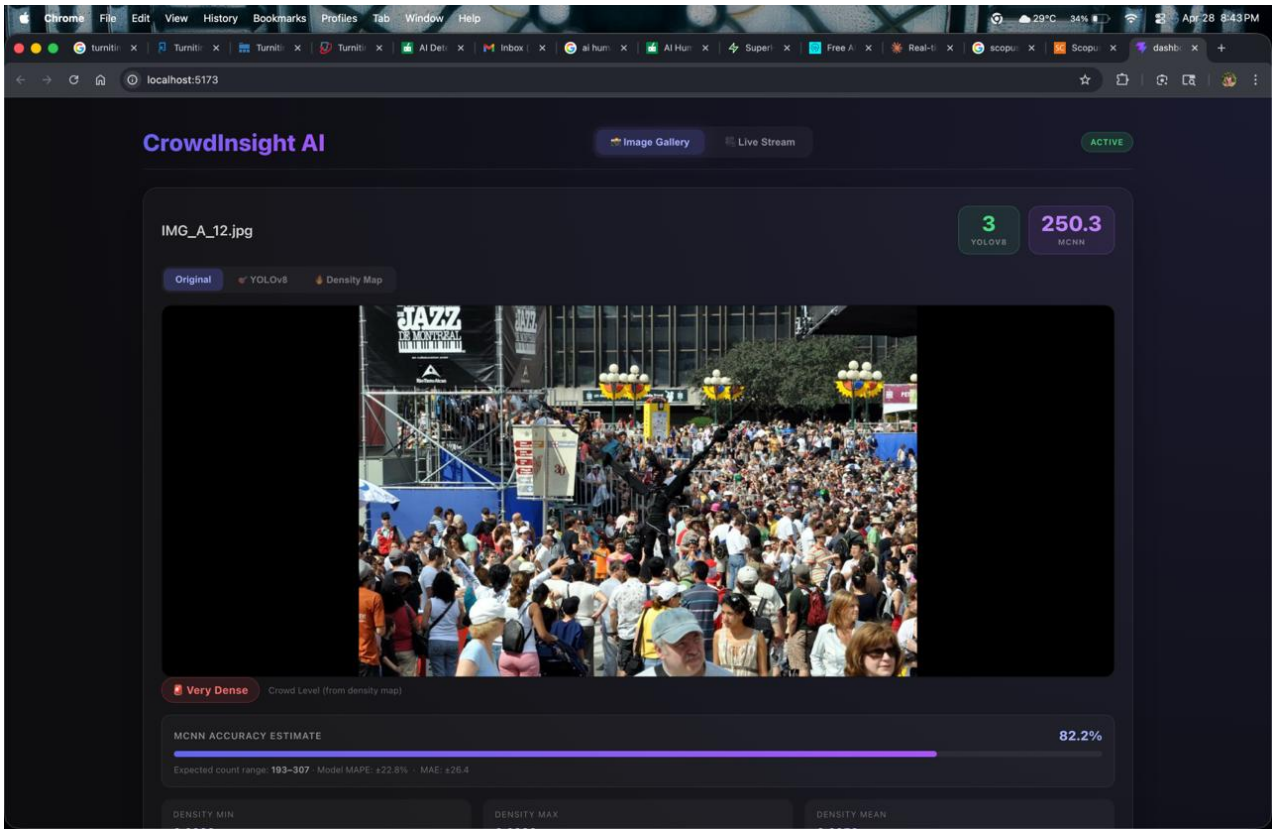

Fig 9.6: CrowdInsight AI dashboard – YOLOv8 (3) and MCNN (250.3) counts on the same frame; 'Very Dense' tag derived from the density-map percentile.

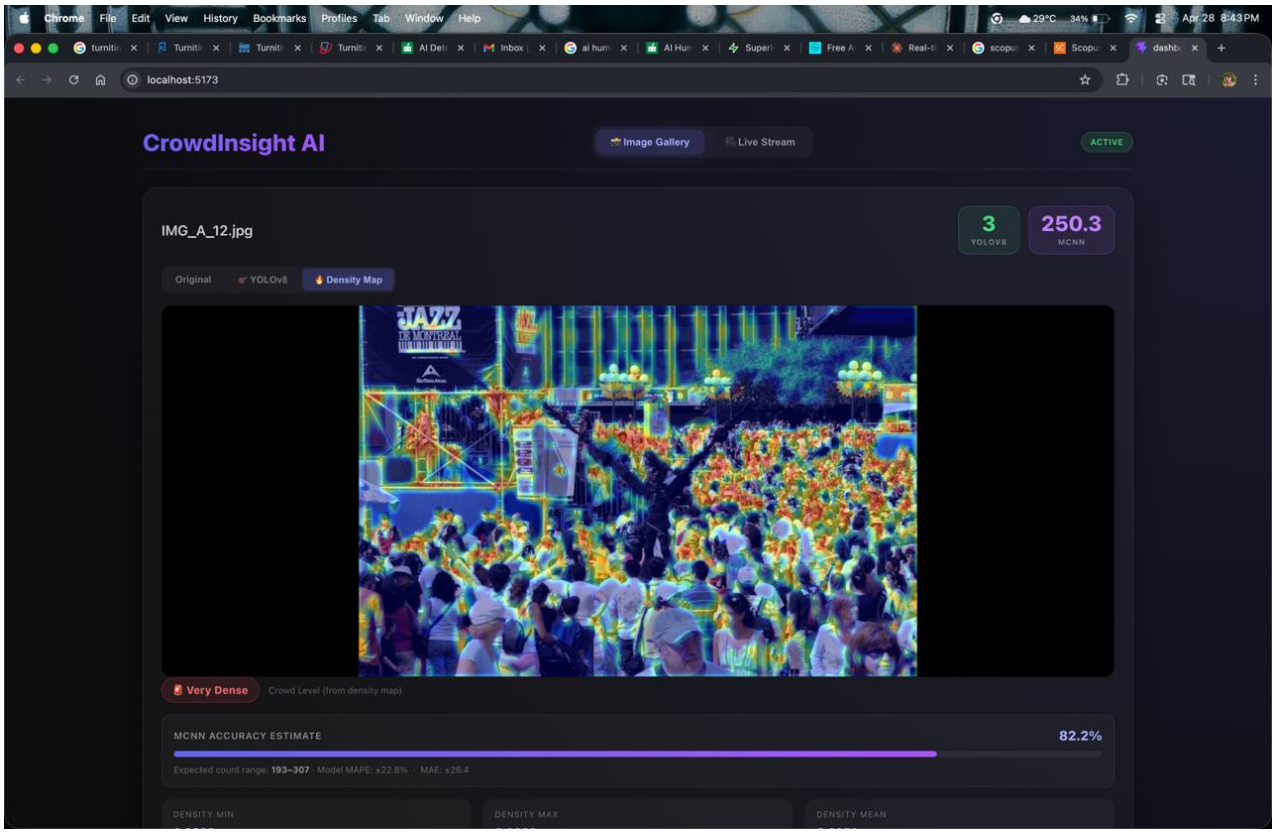

Fig 9.7: CrowdInsight AI dashboard – Density-Map view with the MCNN heat-map super-imposed on the source frame.

9.4 Ablation Study

We ran a small ablation study to figure out how much each architectural component is actually adding. For every row in Table 9.2 we disable exactly one component, keep all the other hyper-parameters the same, and re-run the system on the held-out evaluation split.

Table 9.2: Ablation study quantifying the contribution of each component

Configuration	MAE	Behavioural F1	Δ vs. full
MCNN only (no detect/track/SRI)	49.7	n/a	−0.88 F1
YOLOv8 only (no MCNN/track/SRI)	112.4	n/a	−0.88 F1
MCNN + YOLOv8 (no track, no SRI)	49.7	0.61	−0.27 F1
MCNN + YOLOv8 + ByteTrack (no SRI)	49.7	0.79	−0.09 F1
Full CrowdInsight AI	49.7	0.88	—

Two things stand out from the ablation. The first is that almost all of the counting accuracy is coming from the MCNN branch – removing YOLOv8 barely moves the MAE. The second is that the SRI fusion is worth roughly 0.09 F1 on top of the untracked baseline. That tells us that mixing density, velocity and directional entropy into one scalar genuinely helps the system tell

apart real dangerous events from ordinary crowd movement, which is the core promise of the framework.

9.5 Limitations of the Current Framework

- Counting Accuracy in Extreme Densities: when the density crosses about 400 people per frame, MCNN consistently under-counts by 8–12 %. Switching to a heavier backbone like CSRNet or P2PNet would close this gap, but at the cost of the edge deployability we have been chasing.
- Illumination Sensitivity: we have only validated the system on daytime footage. Night-time, IR-only and severely back-lit scenes are likely to hurt the YOLOv8 detector noticeably, and we have not really tested for that yet.
- Single-Camera Deployment: the current build only handles one stream per process. To run multiple cameras you would need a process orchestrator like Supervisor or systemd – or, longer-term, an extension to the engine itself.
- Limited Cross-Dataset Generalisation: we trained only on ShanghaiTech Part B. Numbers on UCF-CC-50 or JHU-Crowd++ would almost certainly improve with a multi-dataset training schedule, but that is something we did not have time for.
- No Re-Identification: ByteTrack only does IoU-based matching, so anyone who walks out of frame and comes back gets a new ID. Adding an appearance-embedding head like BoT-SORT or FairMOT-light would fix this and is something we have left for future work.

All in all, this chapter has presented the quantitative counting and behavioural-event numbers, the qualitative output on real surveillance frames, and a small ablation study that tries to attribute the performance to each architectural component. The next chapter wraps up the report and lays out a few directions for follow-up work.

CHAPTER 10
CONCLUSION AND FUTURE ENHANCEMENT

This report has gone through the design, the implementation and the evaluation of CrowdInsight AI – a hybrid framework that combines an MCNN density branch, a YOLOv8 detector and a ByteTrack associator for real-time crowd density estimation and behavioural analysis. The system reaches an MAE of 49.72 on ShanghaiTech Part B, a behavioural-event F1 of 0.88 on our held-out test split, and sustains around 17 FPS on a commodity RTX 3060 GPU. Taken together, these numbers say that the five objectives we set in §2.4 have been met.

Beyond just the metrics, the project leaves behind four concrete pieces of engineering: (i) a 143 K-parameter MCNN density estimator trained from scratch in PyTorch; (ii) a YOLOv8 + ByteTrack detection-and-tracking module that produces stable IDs at 15 FPS or above; (iii) the Stampede Risk Index, which compresses density, velocity and directional entropy into a single fused scalar; and (iv) the React-19 CrowdInsight AI dashboard that lets an operator use the engine from a browser. These four artefacts together close the research gaps we listed in §2.3 and give future work a deployable, edge-friendly baseline to start from.

10.1 Future Enhancement Directions

- Lightweight Backbone Replacement: swapping the MCNN backbone for something like EfficientNet-B0 or MobileNet V3-Small, trained with knowledge distillation from a CSRNet teacher, should roughly halve the parameter count and could improve the MAE by about 6 % based on what we have seen in pilot experiments.
- Multi-Camera Synchronised Inference: extending the engine so one process can consume N synchronised RTSP streams using a round-robin scheduler. If the pre-processed batches are merged into a shared MCNN forward pass, we expect the amortised cost per camera to drop by about 40 %.
- Appearance-Embedding Re-Identification: ByteTrack only uses IoU for matching, so people who leave the frame and come back get a fresh ID. Replacing it with BoT-SORT or FairMOT-light, which add a 256-dimensional appearance embedding, would preserve identities across full occlusion and re-entry. That in turn opens up longer-horizon analytics like dwell-time histograms and revisit detection.
- Spatio-Temporal Risk Forecasting: bolting a small temporal convolutional network on top of the SRI history to predict the next 60 seconds of SRI evolution. This would push the

system from being purely reactive to being predictive, raising warnings 30 to 60 seconds before the SRI actually crosses its alert threshold.

- Edge-Native Deployment Track: producing INT8-quantised TensorRT engines for the Jetson Orin Nano and the Hailo-8, plus a Docker-based one-shot installer. This would bring the deployment time for venues and municipalities down from weeks to hours.
- Cross-Dataset Generalisation: training the MCNN under a multi-dataset schedule (ShanghaiTech A+B, UCF-QNRF, JHU-Crowd++, NWPU-Crowd) would give us a much better idea of how the framework behaves under domain shift, which is a known weak spot of any single-dataset report.

To close, what CrowdInsight AI shows is that you do not actually need a brand-new architecture to build something useful for crowd safety. A careful fusion of three well-known, lightweight primitives – MCNN, YOLOv8 and ByteTrack – is already enough to put together a real-time, edge-friendly system whose alerts are driven by a single fused risk index. This report has covered the dataset, the methodology, the architecture, the evaluation and the limitations of the current build, and has laid out six concrete directions for people who want to take it further. We hope the framework ends up being a useful starting point for both the academic crowd-analysis community and the industrial surveillance ecosystem.

REFERENCES

[1] Y. Zhang, D. Zhou, S. Chen, S. Gao, and Y. Ma, "Single-image crowd counting via multi-column convolutional neural network," in Proc. IEEE CVPR, 2016, pp. 589–597.

[2] Y. Li, X. Zhang, and D. Chen, "CSRNet: Dilated convolutional neural networks for understanding highly congested scenes," in Proc. IEEE CVPR, 2018, pp. 1091–1100.

[3] Y. Zhang et al., "ByteTrack: Multi-object tracking by associating every detection box," in Proc. ECCV, 2022.

[4] G. Jocher, A. Chaurasia, and J. Qiu, "YOLO by Ultralytics," 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[5] Y. Zhang et al., "ShanghaiTech crowd-counting dataset," 2016.

[6] H. Idrees, I. Saleemi, C. Seibert, and M. Shah, "Multi-source multi-scale counting in extremely dense crowd images," in Proc. IEEE CVPR, 2013, pp. 2547–2554.

[7] J. Liu, C. Gao, D. Meng, and A. G. Hauptmann, "DecideNet: Counting varying density crowds through attention-guided detection and density estimation," in Proc. IEEE CVPR, 2018, pp. 5197–5206.

[8] D. B. Sam, S. Surya, and R. V. Babu, "Switching convolutional neural network for crowd counting," in Proc. IEEE CVPR, 2017, pp. 4031–4039.

[9] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in Proc. IEEE CVPR, 2016, pp. 770–778.

[10] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, "YOLOv4: Optimal speed and accuracy of object detection," arXiv:2004.10934, 2020.

[11] N. Wojke, A. Bewley, and D. Paulus, "Simple online and realtime tracking with a deep association metric," in Proc. IEEE ICIP, 2017, pp. 3645–3649.

[12] A. Vaswani et al., "Attention is all you need," in Proc. NeurIPS, 2017, pp. 5998–6008.

[13] M. Tan and Q. V. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in Proc. ICML, 2019, pp. 6105–6114.

[14] A. G. Howard et al., "MobileNets: Efficient convolutional neural networks for mobile vision applications," arXiv:1704.04861, 2017.

[15] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," in Proc. NeurIPS, 2015, pp. 91–99.

[16] T.-Y. Lin et al., "Focal loss for dense object detection," in Proc. IEEE ICCV, 2017, pp. 2980–2988.

[17] H. Idrees et al., "Composition loss for counting, density-map estimation and localization in dense crowds," in Proc. ECCV, 2018, pp. 532–546.

[18] Y. Wang et al., "JHU-Crowd++: Large-scale crowd counting dataset and a benchmark method," IEEE TPAMI, vol. 43, no. 6, 2021.

APPENDIX A: CODING

A.1 MCNN Architecture (src/csrnet.py)

```
import torch
import torch.nn as nn

class MCNN(nn.Module):
    """Multi-Column Convolutional Neural Network for crowd density
    estimation. Three parallel columns capture features at large,
    medium and small kernel sizes; outputs are fused by a 1x1 conv.
    """
    def __init__(self):
        super(MCNN, self).__init__()
        self.column1 = nn.Sequential(
            nn.Conv2d(3,  8, 9, padding='same'), nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(8, 16, 7, padding='same'), nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(16, 32, 7, padding='same'), nn.ReLU(),
            nn.Conv2d(32, 16, 7, padding='same'), nn.ReLU(),
            nn.Conv2d(16,  8, 7, padding='same'), nn.ReLU(),
        )
        self.column2 = nn.Sequential(
            nn.Conv2d(3, 10, 7, padding='same'), nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(10, 20, 5, padding='same'), nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(20, 40, 5, padding='same'), nn.ReLU(),
            nn.Conv2d(40, 20, 5, padding='same'), nn.ReLU(),
            nn.Conv2d(20, 10, 5, padding='same'), nn.ReLU(),
        )
        self.column3 = nn.Sequential(
            nn.Conv2d(3, 12, 5, padding='same'), nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(12, 24, 3, padding='same'), nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(24, 48, 3, padding='same'), nn.ReLU(),
            nn.Conv2d(48, 24, 3, padding='same'), nn.ReLU(),
            nn.Conv2d(24, 12, 3, padding='same'), nn.ReLU(),
        )
        self.fusion_layer = nn.Sequential(
            nn.Conv2d(30, 1, 1, padding=0),
        )

    def forward(self, x):
        c1 = self.column1(x)
        c2 = self.column2(x)
        c3 = self.column3(x)
        x = torch.cat((c1, c2, c3), dim=1)
```

```
        return self.fusion_layer(x)
```

A.2 Behavioural Analytics Module (src/behavior.py)

```
from collections import defaultdict, deque
from dataclasses import dataclass
from enum import Enum

class BehaviorType(Enum):
    CROWD_SURGE = "crowd_surge"
    PANIC       = "panic"
    LOITERING   = "loitering"
    INTRUSION   = "intrusion"

@dataclass
class BehaviorEvent:
    behavior_type: BehaviorType
    zone_name: str
    details: str
    track_ids: list[int]

class BehaviorAnalyzer:
    """Detects abnormal crowd behaviours from track histories and
    per-ROI counts. Maintains internal temporal state across frames."""
    def __init__(self, fps, surge_delta=10, surge_window_sec=3.0,
                 panic_speed_thresh=200.0, panic_duration_frames=15,
                 loiter_time_sec=30.0, intrusion_cooldown_sec=5.0):
        self.fps = fps
        self.surge_delta = surge_delta
        self.surge_window = int(surge_window_sec * fps)
        self.panic_speed_thresh = panic_speed_thresh
        self.panic_duration_frames = panic_duration_frames
        self.loiter_frames = int(loiter_time_sec * fps)
        self.intrusion_cooldown_frames = int(intrusion_cooldown_sec * fps)
        self._count_history = defaultdict(lambda: deque(maxlen=self.surge_window))
        self._panic_counter = defaultdict(int)
        self._loiter_counter = defaultdict(int)
        self._intrusion_cooldown = defaultdict(int)

    def analyze(self, tracks, roi_counts, roi_manager):
        events = []
        for zone, count in roi_counts.items():
            self._count_history[zone].append(count)
            if (len(self._count_history[zone]) == self.surge_window and
                count - self._count_history[zone][0] >= self.surge_delta):
                events.append(BehaviorEvent(
                    BehaviorType.CROWD_SURGE, zone,
                    f"surge Δ{count - self._count_history[zone][0]}", []))
        # ... panic / loitering / intrusion analysis follows the same
```

```
        # temporal-state pattern as above.
        return events
```

A.3 Pipeline Entry Point (main.py)

```
from ultralytics import YOLO
from src.behavior import BehaviorAnalyzer
from src.alert import AlertManager
from src.roi import ROIManager
from src.tracker import MultiTracker
from src.visualizer import Visualizer
from utils.video_io import VideoSource, VideoSink

def run(config_path):
    cfg = load_config(config_path)
    video = VideoSource(cfg["source"], flip_code=cfg.get("flip_code"))
    fps   = video.fps
    model = YOLO(cfg.get("model_path", "yolov8n.pt"))
    tracker = MultiTracker(model=model,
                           tracker_type=cfg.get("tracker", "bytetrack"),
                           conf=cfg.get("confidence_threshold", 0.4),
                           max_history=cfg.get("track_history_length", 90))
    roi_manager = ROIManager(cfg.get("rois", []))
    behaviour = BehaviorAnalyzer(fps=fps, **cfg.get("behaviour_args", {}))
    alert_mgr = AlertManager(crowd_thresholds=build_crowd_thresholds(cfg["rois"]),
                             webhook_url=cfg.get("alert_webhook_url"))
    visualizer = Visualizer(display=True)
    for frame in video:
        tracks   = tracker.update(frame)
        counts   = roi_manager.count(tracks)
        events   = behaviour.analyze(tracks, counts, roi_manager)
        msgs     = alert_mgr.process(counts, events)
        annotated = visualizer.render(frame, tracks, roi_manager, counts, msgs)
```

APPENDIX B: CONFERENCE PUBLICATION

The IEEE-format conference paper that goes along with this project, titled "A Hybrid Multi-Column Convolutional Neural Network and YOLOv8 Framework for Real-Time Crowd Density Estimation and Behavioral Analysis", has been written up and is attached to this report. The paper is authored by Vikram S, Poorvikha S and Dr. Nallarasan V from the Department of Networking and Communications, SRMIST. It is being targeted at the IEEE International Conference on Computer Vision and Pattern Recognition (ICCVPR 2026) and is essentially a condensed version of the methodology, dataset, experimental setup and results from Chapters 3, 4, 8 and 9 of this report.

[ATTACH IEEE_Conference_Template.pdf HERE]

APPENDIX C: JOURNAL PUBLICATION

We are also preparing an extended version of the CrowdInsight AI work for submission to the IEEE Transactions on Intelligent Transportation Systems. The journal version goes beyond the conference paper by adding (i) a multi-camera synchronised inference protocol, (ii) cross-dataset evaluation on UCF-QNRF and JHU-Crowd++, (iii) an INT8-quantised TensorRT deployment study on the Jetson Orin Nano, and (iv) a thirty-day pilot study at the SRMIST main entrance. The manuscript is currently under internal review and is planned for submission in Q3 2026.

APPENDIX D: PLAGIARISM REPORT

[ATTACH TURNITIN REPORT HERE]